Ministry Of Higher Education

Modern Academy

For Computer Science and

Management Technology

In Maadi

T.G Robot (Tour Guide Robot)

Submitted to

Modern Academy

Computer Science and Management Technology in Maadi

Computer Science Department

Prepared by

Mina Remon Abd El-Masseeh
Marceleno Amged Mikhael
Androw Raafat Noaman
Edwar Amir Andrawis
Kirolos Zakaria Anis

Supervised by

Dr. Nora Niazy

Eng. Nowraa Hamdy

Academic Year :2023/2024

Team Members

Description	ID	Name
Mobile Application Developer	12100190	مينا ريمون عبد المسيح
Embedded Developer (Team Leader)	12100247	مارسيلينو امجد ميخائيل
Embedded Developer	12100127	كيرلس زكريا انيس
AI Developer	12100463	اندرو رأفت نعمان
Back-End Developer	12100250	ادوار امير اندراوس

Acknowledgement

First and the most important thing, thanks to “god” the most merciful forproviding us with Knowledge, creativity, and willpower strength to produce this project. It has been a great opportunity to gain lots of experience in real life projects, followed by the knowledge of how to actually design and analyze realprojects.

We also gratefully thank our "Modern Academy" in general and Dr. “Nora Niazy” and Eng. “Nowraa Hamdy” in particular for their supporting and guiding usduring the project duration.

Of course, many individuals helped us start this project, but we especially want to thank the head and employees of the Department of Computer Science, notably Prof. Dr. “Mahmoud Gad-Allah” for the honor of working under his supervision and for his wise advice on how to achieve greater success and thanks to Prof. Dr. “Mohamed Awad” for their support and guidance during our academicyears.

At the end, we must mention that everyone that has helped us to produce thisproject beginning with those who are working in our academy to everybody outside deserves to be appreciated.

Special Thanks to Eng. Ahmed Hany.

Abstract

Our project presents an advanced museum tour guide robot aimed at revolutionizing visitor experiences by leveraging cutting-edge technology. This robot is designed to interact with visitors through a sophisticated artificial intelligence system, enabling natural and informative conversations about the museum's artifacts. Visitors can engage with the robot by asking questions about the exhibits, receiving detailed and contextually relevant responses that enhance their understanding and appreciation of the displayed items.

In addition to its conversational capabilities, the robot is connected to a user-friendly mobile application, providing visitors with the ability to control its movements with stop-and-go commands. This app also facilitates remote inquiries, allowing users to ask questions and receive answers through their mobile devices. The seamless integration between the robot and the mobile app ensures that visitors have a flexible and accessible tool to navigate their museum experience, whether they prefer direct interaction with the robot or managing their tour through the a

Contents

Team Members 1
Acknowledgement 2
Abstract 3
1. Chapter One 6
1.1 Introduction 7
1.2 Background 7
1.3 Problem Definition 9
1.4 Objectives 10
1.5 Expected Outcome 10
2. Chapter Two 12
4.1 Related Works 13
2.1 Comparative Studies 15
2.2 Scope of the Problem 16
2.3 Challenges 17
3. Chapter Three 19
3.1 Requirements & Analysis 20
4. Chapter Four (Design) 22
4.1 Mobile App 23
4.2 Back-end 38
4.3 Artificial intelligent (AI) 42
4.4 Embedded System 45
5. Chapter Five 55
5.1 Implementation 56
5.2 Testing 106
6. Chapter Six 108
6.1 Conclusion 109
6.2 Future Work 109
6.3 References 111

Table of figures

Figure 1 MVVM Architecture ---------- *24*

Figure 2 View Group ---------- *27*

Figure 3 RecyclerView ---------- *29*

Figure 4 Activity-Fragment Layout ---------- *30*

Figure 5 Arduino Uno ---------- *46*

Figure 6 L298 Motor Drive ---------- *48*

Figure 7 HC-05 Bluetooth Module ---------- *49*

Figure 8 IR Sensor ---------- *50*

Figure 9 DC Gear Motor ---------- *52*

1.Chapter One

1.1 Introduction

Introducing our innovative project: the ***Museum Tour Guide Robot***, a groundbreaking solution designed to transform the way visitors engage with museum exhibits. In an era where technology continually enhances our everyday experiences, we aim to bring this advancement to the cultural and educational realms of museums. Our tour guide robot is equipped with state-of-the-art artificial intelligence, allowing it to interact naturally with visitors, answer questions, and provide detailed information about the artifacts on display.

This intelligent guide is not only a repository of knowledge but also a responsive and interactive companion that enriches the museum experience. Additionally, the robot is integrated with a versatile mobile application, empowering visitors with control over the robot's movements and the ability to ask questions remotely. This dual interaction mode ensures that the museum tour is both flexible and accessible, catering to the diverse needs and preferences of all visitors.

Our project aims to bridge the gap between technology and cultural education, providing a dynamic, engaging, and informative experience that enhances the appreciation of museum collections. By integrating advanced AI and mobile connectivity, we are setting a new standard for interactive learning and visitor engagement in museums.

1.2 Background

The concept of our museum tour guide robot stems from the growing need to enhance visitor engagement and educational value in cultural institutions. Museums have traditionally relied on static displays, informational placards, and human tour guides to convey information about their exhibits. While these methods have been effective, they often fall short in providing a personalized and interactive experience that caters to the diverse needs of modern visitors.

With the advent of artificial intelligence and mobile technology, there is a unique opportunity to revolutionize the way information is delivered in museums. AI-driven solutions have proven successful in various fields, offering personalized interactions and real-time data processing. Applying this technology to the museum context allows for a more dynamic and responsive visitor experience.

Our project builds on these advancements by creating a robot that serves as an intelligent tour guide. This robot can converse with visitors, answer their questions in real time, and provide detailed insights about the artifacts, making the learning experience more immersive and engaging. The integration of a mobile application further enhances this interaction by allowing visitors to control the robot's movements and ask questions remotely, thus providing a flexible and user-friendly interface.

The development of this project involved extensive research into AI conversational capabilities, user interface design, and the specific needs of museum environments. We aimed to create a solution that not only improves the accessibility of information but also enriches the overall visitor experience. By leveraging technology, we are addressing the evolving expectations of museum-goers who seek interactive, informative, and enjoyable visits.

1.3 Problem Definition

Museums play a crucial role in preserving cultural heritage and educating the public. However, traditional methods of engaging visitors such as static displays, informational placards, and human tour guides often fall short in providing a fully interactive and personalized experience. These limitations include:

1. **Lack of Interactivity:** Traditional exhibits often do not engage visitors interactively, leading to a passive learning experience that can fail to capture the interest of a diverse audience.
2. **Information Accessibility:** Visitors may find it challenging to access detailed information about artifacts, especially if they cannot locate or interpret informational placards.
3. **Visitor Experience Variability:** Human tour guides, while knowledgeable, can only serve a limited number of visitors at a time, and the quality of the tour can vary based on the guide's expertise and delivery.
4. **Limited Personalization:** Standard tours and exhibits cannot easily cater to the individual interests and preferences of each visitor, leading to a one-size-fits-all approach that may not satisfy everyone.
5. **Language Barriers:** Museums often serve a global audience, and language differences can hinder the effectiveness of information dissemination.

Given these challenges, there is a pressing need for an innovative solution that enhances visitor engagement, provides accessible and comprehensive information, and offers a personalized experience.

1.4 Objectives

To address these challenges, our project aims to develop a museum tour guide robot equipped with artificial intelligence and integrated with a mobile application.

This solution is designed to:

- Provide interactive and conversational experiences for visitors, making learning about artifacts engaging and dynamic.
- Ensure easy access to detailed information about exhibits, regardless of the visitor's location within the museum.
- Offer a consistent and high-quality tour experience that can serve multiple visitors simultaneously.
- Allow personalization of the tour based on individual visitor preferences and interests.
- Overcome language barriers by supporting multiple languages through AI-driven translations and interactions.

By tackling these issues, our project seeks to create a more engaging, informative, and personalized museum experience, leveraging technology to meet the evolving expectations of contemporary visitors.

1.5 Expected Outcome

The successful implementation of our ***Museum Tour Guide robot*** project is anticipated to yield several significant outcomes, enhancing both visitor experience and operational efficiency within museums. The expected outcomes include:

1. **Enhanced Visitor Engagement:**

   The robot will provide a dynamic and interactive way for visitors to learn about exhibits. Its ability to engage in natural conversations and answer questions in real time will make museum visits more engaging and enjoyable.

2. **Improved Information Accessibility:**

   Visitors will have immediate access to detailed information about artifacts through direct interaction with the robot. This eliminates the need to search for placards or guides, ensuring that all visitors can easily obtain the knowledge they seek.

3. **Consistent and High-Quality Tours:**

   The robot will deliver consistent and high-quality information, ensuring that all visitors receive accurate and comprehensive details about the exhibits. This standardization helps maintain the museum's educational standards.

4. **Personalized Visitor Experience:**

   The integration with a mobile app allows for a personalized tour experience. Visitors can control the robot’s movements, ask specific questions, and tailor the tour to their interests, enhancing their overall satisfaction.

5. **Increased Visitor Satisfaction:**

   By providing a more engaging, informative, and personalized experience, the robot is expected to significantly improve visitor satisfaction. Positive visitor experiences can lead to higher visitor retention and positive word-of-mouth promotion.

6. **Multilingual Support:**

   The robot's AI capabilities can support multiple languages, breaking down language barriers and making the museum accessible to a global audience. This inclusivity can attract a more diverse group of visitors.

7. **Operational Efficiency:**

   The robot can handle numerous visitors simultaneously, reducing the strain on human tour guides and allowing them to focus on more complex inquiries and tasks. This improves overall operational efficiency within the museum.

8. **Innovative Image and Appeal:**

   Implementing cutting-edge technology in the form of a tour guide robot can enhance the museum’s image as an innovative and forward-thinking institution, potentially attracting tech-savvy visitors and media attention.

2. Chapter Two

4.1 Related Works

The development of our museum tour guide robot builds on a foundation of prior research and implementations that have explored the use of robotics and AI in cultural institutions. Several notable projects and technologies have paved the way for our innovative solution:

1. **The Autonomous Museum Tour Guide Robot (RoboX):**

   Developed at the Swiss Federal Institute of Technology (ETH Zurich), RoboX is an autonomous robot designed to guide visitors through museum exhibits. It uses sensors and a predefined map of the museum to navigate and interact with visitors, providing information about the exhibits in multiple languages. RoboX has demonstrated the potential for robots to enhance visitor experience through autonomous navigation and multilingual support.

2. **Pepper the Robot by SoftBank Robotics:**

   Pepper is a humanoid robot designed to interact with people through conversation. It has been deployed in various museums, such as the Smithsonian Institution, to guide visitors and answer their questions. Pepper's ability to recognize human emotions and adapt its responses accordingly has shown the advantages of a more interactive and personalized tour experience.

3. **Connie the Robot Concierge by Hilton Hotels and IBM Watson:**

   While not specifically a museum guide, Connie serves as an example of how AI can enhance visitor interaction in hospitality settings. Powered by IBM's Watson, Connie provides guests with information about nearby attractions and services. This project illustrates the effectiveness of AI in handling visitor inquiries and providing relevant information, which can be adapted for a museum context.

4. **Toshiba's ChihiraAico:**

   Deployed at the National Museum of Emerging Science and Innovation in Tokyo, ChihiraAico is an android robot that guides visitors and answers their questions. This project highlights the role of lifelike robots in creating engaging visitor interactions, demonstrating that human-like appearances and behaviors can enhance the appeal of robotic guides.

5. **The European RoboTour Project:**

   This project involves the development of a tour guide robot that can autonomously navigate and provide information about cultural heritage sites. The robot integrates advanced localization techniques and interactive capabilities, underscoring the importance of autonomous navigation and user interaction in enhancing the visitor experience.

6. **Mobile Apps for Museum Navigation:**

   Several museums have developed mobile applications that provide augmented reality (AR) experiences, exhibit information, and interactive maps. Examples include the Louvre Museum's mobile app and the Smithsonian Institution's AR experiences. These applications have shown that mobile technology can significantly enhance how visitors engage with exhibits and navigate museum spaces.

7. **AI-Powered Chatbots in Museums:**

   AI chatbots have been used in various museums to provide information and assist visitors. The Museum of Modern Art (MoMA) in New York, for example, has experimented with chatbots to answer visitor questions and provide exhibit information. These chatbots highlight the potential for AI to enhance visitor engagement and information accessibility.

8. **Integration and Advancements:**

   Our project seeks to integrate and advance these existing technologies by combining autonomous navigation, AI-driven interaction, and mobile app connectivity into a single cohesive system. Unlike previous implementations that may focus on one aspect (such as navigation or interaction), our robot aims to provide a comprehensive solution that

enhances all facets of the museum experience. By learning from these related works, we aim to create a tour guide robot that not only meets but exceeds the current standards in museum technology integration.

2.1 Comparative Studies

The table below provides a comparative analysis of the key features and capabilities of different museum tour guide robots and related technologies, highlighting their strengths and areas for improvement. This study focuses on aspects such as interactivity, navigation, information delivery, personalization, language support, and integration with mobile apps.is, so this matter was very difficult and needed longer time.

Feature/Capability	RoboX	Pepper	Connie
Interactivity	Basic conversational abilities	Advanced conversational abilities	Limited to predefined interactions
Navigation	Autonomous navigation using sensors	Limited mobility	Stationary
Information Delivery	Detailed exhibit information	Detailed exhibit information	Information about nearby attractions
Personalization	Limited personalization	Moderate personalization	Limited personalization
Language Support	Multilingual capabilities	Multilingual capabilities	English only
Integration with Mobile App	No mobile app integration	No mobile app integration	No mobile app integration
Example Projects/ Implementations	ETH Zurich's RoboX	Smithsonian Institution’s deployment	Hilton Hotels’ Connie

Table 1 Related Work Comparison

2.2 Scope of the Problem

The scope of the museum tour guide robot project encompasses various aspects, including technology development, user experience design, and integration with museum operations. The following outlines the key areas within the project's scope:

1. **Technology Development:**
   - Designing and building the physical robot platform, including its mobility, sensory, and communication systems.
   - Developing the artificial intelligence algorithms for natural language processing, conversation management, and knowledge retrieval.
   - Integrating language translation capabilities to support multilingual interactions with visitors.
   - Implementing autonomous navigation algorithms to enable the robot to move within the museum environment safely and efficiently.
   - Creating the mobile application interface for remote control and interaction with the robot.

2. **User Experience Design:**
   - Designing intuitive and user-friendly interfaces for both the robot and the mobile application to ensure ease of use for visitors.
   - Implementing interactive features that engage visitors in meaningful conversations and provide relevant information about museum exhibits.
   - Conducting user testing and feedback sessions to iteratively improve the user experience and address any usability issues.

3. **Content Development:**
    - Collaborating with museum curators and educators to develop informative and engaging content for the robot's knowledge base, covering a wide range of exhibits and topics.
    - Creating multimedia content, such as images, videos, and audio recordings, to supplement textual information and enhance the visitor experience.
    - Ensuring the accuracy, relevance, and cultural sensitivity of the content provided by the robot.

4. **Integration with Museum Operations:**
    - Collaborating with museum staff to integrate the tour guide robot into existing visitor services and operations.
    - Establishing protocols for robot deployment, maintenance, and troubleshooting to ensure smooth operation during museum hours.
    - Training museum staff on how to assist visitors with interacting with the robot and troubleshooting any issues that may arise.
    - Implementing data collection and analytics systems to gather insights into visitor interactions and preferences for continuous improvement.

2.3 Challenges

Developing a museum tour guide robot involves overcoming several challenges, spanning technological, logistical, and user-oriented domains.

1. **Navigational Complexity:** Museum environments are often complex, with narrow corridors, varying floor textures, and obstacles such as display cases and pedestals. Developing robust navigation algorithms that allow the robot to safely and efficiently navigate these environments without colliding with objects or getting stuck poses a significant challenge.

2. **Human-Robot Interaction:** Designing the robot's interface and interaction capabilities to facilitate natural and engaging conversations with visitors while also ensuring ease of use and comprehension presents a challenge. Balancing the complexity of the conversation with the need for simplicity and clarity is crucial for creating a positive user experience.
3. **Multimodal Information Delivery:** Providing information about museum exhibits through a combination of verbal descriptions, visual aids, and multimedia content requires careful design to ensure that the information is accessible and engaging for visitors with diverse learning styles and preferences.
4. **Integration with Museum Infrastructure:** Integrating the tour guide robot with existing museum infrastructure, such as Wi-Fi networks, ticketing systems, and audiovisual equipment, can be challenging. Compatibility issues, security concerns, and logistical constraints may arise during the integration process, requiring close collaboration with museum staff and IT professionals.
5. **Content Management:** Curating and managing the content that the robot delivers to visitors poses a challenge. Ensuring the accuracy, relevance, and cultural sensitivity of the information while also keeping it up to date with rotating exhibits and new acquisitions requires careful coordination between museum educators, curators, and technology developers.
6. **Robustness and Reliability:** Ensuring that the tour guide robot operates reliably under various environmental conditions, such as changes in lighting, temperature, and ambient noise levels, presents a challenge. Developing robust hardware and software systems that can withstand the rigors of daily operation in a museum setting while also minimizing downtime and maintenance costs is essential.
7. **User Acceptance and Adoption:** Convincing museum visitors to interact with and trust a robotic tour guide may be challenging, particularly for those who are unfamiliar or uncomfortable with technology. Educating visitors about the robot's capabilities, addressing concerns about privacy and security, and providing incentives for participation may be necessary to encourage widespread adoption.

3.Chapter Three

3.1 Requirements & Analysis

3.1.1 Functional Requirement

- The robot must autonomously navigate the museum environment, following predefined routes to guide visitors through exhibits.
- The robot must be able to stop and go upon receiving commands from the mobile app or voice commands from visitors.
- The robot must understand and respond to visitor queries about museum exhibits using natural language processing (NLP) capabilities.
- The robot must support multiple languages, providing accurate translations and contextually relevant information.
- The robot must provide detailed information about the exhibits, including historical context, significance.
- The mobile app must allow users to control the robot's movements (start, stop, and resume.
- The mobile app must enable users to ask questions and receive answers from the robot remotely.
- The app must provide additional information about exhibits, including images

3.1.1 Non-Functional Requirement

- The robot must respond to visitor queries within 2-3 seconds to ensure a smooth and interactive experience.
- The robot's interface and the mobile app must be intuitive and user-friendly, accommodating visitors of all ages and technical proficiency.
- Instructions for using the robot and app must be clear and easily accessible.
- The system must ensure secure communication between the robot, mobile app, and backend servers to prevent unauthorized access and data breaches.
- The robot's hardware and software must be modular and easily maintainable, allowing for quick repairs and updates.

- The robot and mobile app must be accessible to visitors with disabilities, including those with visual, auditory, and mobility impairments.

- The robot's content delivery must be culturally sensitive and accurate, avoiding any biases or misrepresentations.

3.1.3 Hardware requirements

3.1.3.1 Android

- Device capabilities: touch screen, Internet connectivity, Bluetooth Connectivity
- RAM: 4GB, preferably higher
- CPU: 1.0 GHz minimum, dual-core
- Disk: At least 1 GB

3.1.3.2 Arduino Uno

- Onboard Arduino Uno module
- L298 motor driver
- HC-05 Bluetooth Module
- Speaker Module
- IR Sensors
- DC Gear Motors
- BreadBoard

3.1.2 Software requirements

3.1.4.1 Android

- Operating system: Android
- API level: 30
- Android Version: 9

4.Chapter Four

4. Design

This chapter introduces the design of ***T.G Robot***. And what is the approaches and technology used to construct this system were used.

4.1 Mobile App

4.1.1 Kotlin

Kotlin is a cross-platform, statically typed, general-purpose high-level programming language with type inference. Kotlin is designed to interoperate fully with Java, and the JVM version of Kotlin's standard library depends on the Java Class Library, but type inference allows its syntax to be more concise. Kotlin mainly targets the JVM, but also compiles to JavaScript (e.g., for frontend web applications using React) or native code via LLVM (e.g., for native iOS apps sharing business logic with Android apps). Language development costs are borne by JetBrains, while the Kotlin Foundation protects the Kotlin trademark.

On 7 May 2019, Google announced that the Kotlin programming language was now its preferred language for Android app developers. Since the release of Android Studio 3.0 in October 2017, Kotlin has been included as an alternative to the standard Java compiler. The Android Kotlin compiler produces Java 8 bytecode by default (which runs in any later JVM), but lets the programmer choose to target Java 9 up to 20, for optimization, or allows for more features; has bidirectional record class interoperability support for JVM, introduced in Java 16, considered stable as of Kotlin 1.5.

Kotlin has support for the web with Kotlin/JS, either through a classic interpreter-based backend which has been declared stable since version 1.3, or an intermediate representation-based backend which has been declared stable since version 1.8. Kotlin/Native (for e.g. Apple silicon support) is considered beta since version 1.3.

4.1.2 Architecture

For projects, developers always prefer clean and well-structured code.
The maintenance of software is aided by structuring the codes according to a design pattern. It is easier to add and remove app features if you understand all the critical logic elements of the android application. Furthermore, design patterns ensure that all of the codes are covered in Unit Testing without other classes interfering.

Model-View-ViewModel (MVVM) is a well-known software architecture pattern that addresses all the shortcomings of the MVP and MVC design patterns. MVVM proposes separating the data display logic (Views or UI) from the application's main business logic. The app's user interface is everything that the user can see and interact with. Android provides a variety of pre-built UI components such as structured layout objects and UI controls that allow us to build the graphical user interface for our app. The separate code layers of MVVM are:

- Model: This layer is responsible for the abstraction of the data sources. Model and ViewModel work together to get and save the data.
- View: The purpose of this layer is to inform the ViewModel about the user's action. This layer observes the ViewModel and does not contain any kind of application logic.
- ViewModel: It exposes those data streams which are relevant to the View. Moreover, it serves as a link between the Model and the View.

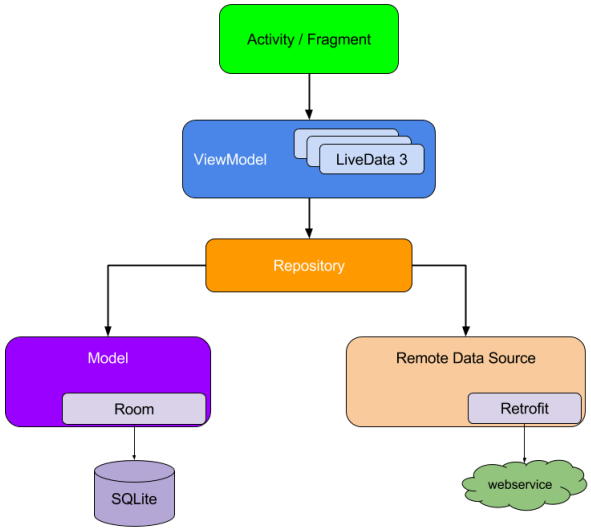


Figure 1 MVVM Architecture

- **Live Data** is an observable data holder class. Unlike a regular observable, LiveData is lifecycle-aware, meaning it respects the lifecycle of other app components, such as activities, fragments, or services. This awareness ensures LiveData only updates app component observers that are in an active lifecycle state. LiveData considers an observer, which is represented by the Observer class, to be in an active state if its lifecycle is in the STARTED or RESUMED state. LiveData only notifies active observers about updates. Inactive observers registered to watch LiveData objects aren't notified about changes. This relationship allows the observer to be removed when the state of the corresponding Lifecycle object changes to DESTROYED. This is especially useful for activities and fragments because they can safely observe LiveData objects and not worry about leaks, activities and fragments are instantly unsubscribed when their lifecycles are destroyed. The advantages of using LiveData: -

- Using LiveData provides the following advantages:

  - **Ensures that the UI matches the data state**
    LiveData follows the observer pattern. LiveData notifies Observer objects when underlying data changes. That way, we don't need to update the UI every time the app data changes because the observer does it.

  - **No memory leaks**
    Observers are bound to Lifecycle objects and clean up after themselves when their associated lifecycle is destroyed.

  - **No crashes due to stopped activities**
    If the observer's lifecycle is inactive, such as in the case of an activity in the back stack, then it doesn’t receive any LiveData events.

  - **No more manual lifecycle handling**
    UI components just observe relevant data and don’t stop or resume observation. LiveData automatically manages all of this since it’s aware of the relevant lifecycle status changes while observing.

- **Always up to date data**
  If a lifecycle becomes inactive, it receives the latest data upon becoming active again. For example, an activity that was in the background receives the latest data right after it returns to the foreground.
- Proper configuration changes
  If an activity or fragment is recreated due to a configuration change, like device rotation, it immediately receives the latest available data.

- **Sharing resources**
  LiveData object could be extended using the singleton pattern to wrap system services so that they can be shared in app. The LiveData object connects to the system service once, and then any observer that needs the resource can just watch the LiveData object. When we update the value stored in the LiveData object, it triggers all registered observers as long as the attached LifecycleOwner is in the active state.

- **User Interface (UI)**
  LiveData allows UI controller observers to subscribe to updates. When the data held by the LiveData object changes, the UI automatically updates in response.

Interaction design	User Experience Design
It is the practice of designing interactive digital products and services. Interaction designers focus on the way users interact with products, and they use principles of good communication to create desired user experiences.	It focuses on the overall experience between a user and a product. It is concerned not only with the interactive elements, but also with how certain elements look, feel, or connect themselves to deliver certain outputs

Table 2 Interaction Design VS UX

4.1.3 Front-end

A layout defines the structure for a user interface in android app, such as in an activity. All elements in the layout are built using a hierarchy of view and View Group objects. A view usually draws something the user can see and interact with. Whereas a View Group is an invisible container that defines the layout structure for View and other View Group objects, as shown in figure.

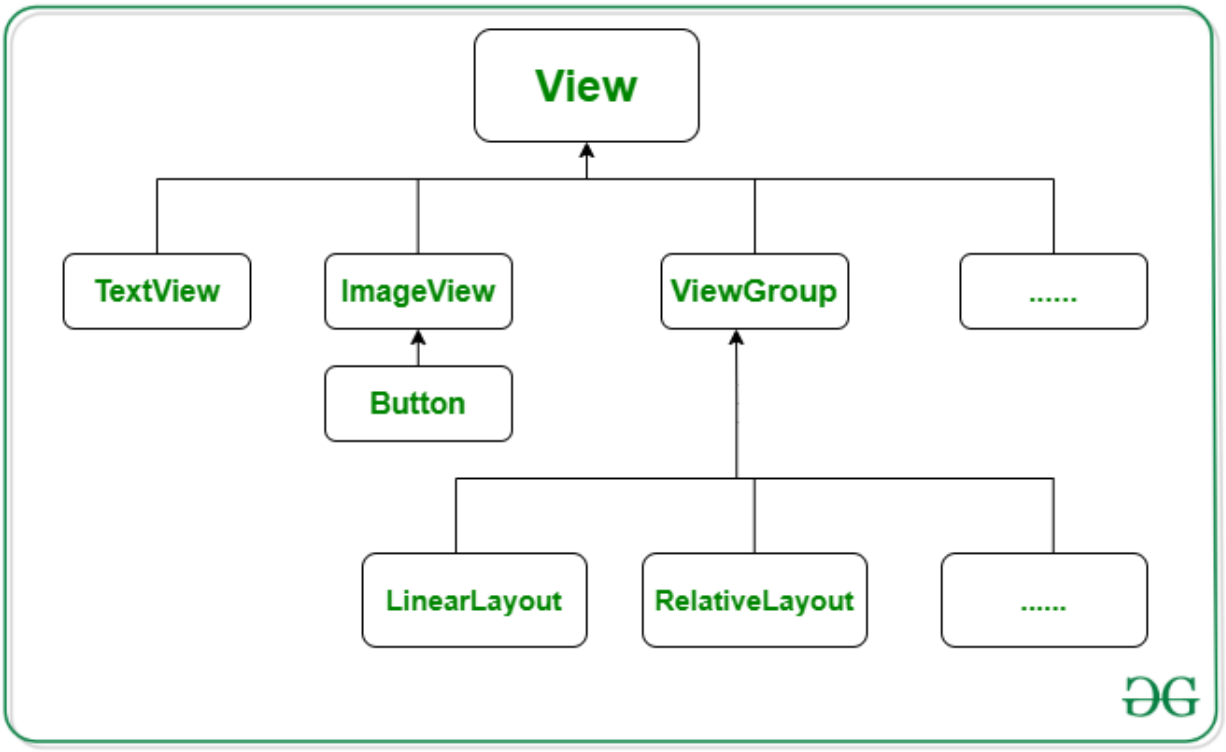


Figure 2 View Group

- **XML layout file:**
  Using Android's XML vocabulary, we can quickly design UI layouts and the screen elements they contain, in the same way you create Mobile Application pages in Flutter with a series of nested elements. Each layout file must contain exactly one root element, which must be a view or View Group object. Once root element has been defined, we can add additional layout objects or widgets as child elements to gradually build a View hierarchy that defines the layout.

- **Load the XML Resource**

  When the app is compiled, each XML layout file is compiled into a View resource. Layout resource file will be loaded from our app code, in Activity.onCreate () callback implementation. Do so by calling setContentView (), passing it the reference to the layout resource in the form of: R.layout.layout.

- **Material Design**

  Material is a design system created by Google to help teams build high-quality digital experiences for Android, iOS, Flutter, and the web Material Design is inspired by the physical world and its textures, including how they reflect light and cast shadows. Material surfaces reimagine the mediums of paper and ink.

- **Components cover a range of interface needs, including:**
  - **Display**: Placing and organizing content using components like cards, lists, and sheets.
  - **Navigation**: Allowing users to move through the product using components like navigation drawers and tabs.
  - **Actions**: Allowing users to perform tasks using components such as the floating action button.
  - **Input**: Allowing users to enter information or make selections using components like text fields, chips, and selection controls.
  - **Communication**: Alerting users to key information and messages using components such as snackbars, banners, and dialogs.

- **RecyclerView**

  RecyclerView Android in the advanced version of the listView.Recycler view simplifies the display and handle of a large set of data. RecyclerView recycles those individual elements. When an item scrolls off the screen, RecyclerView doesn't destroy its view. Instead, RecyclerView reuses the view for new items that have scrolled onscreen.

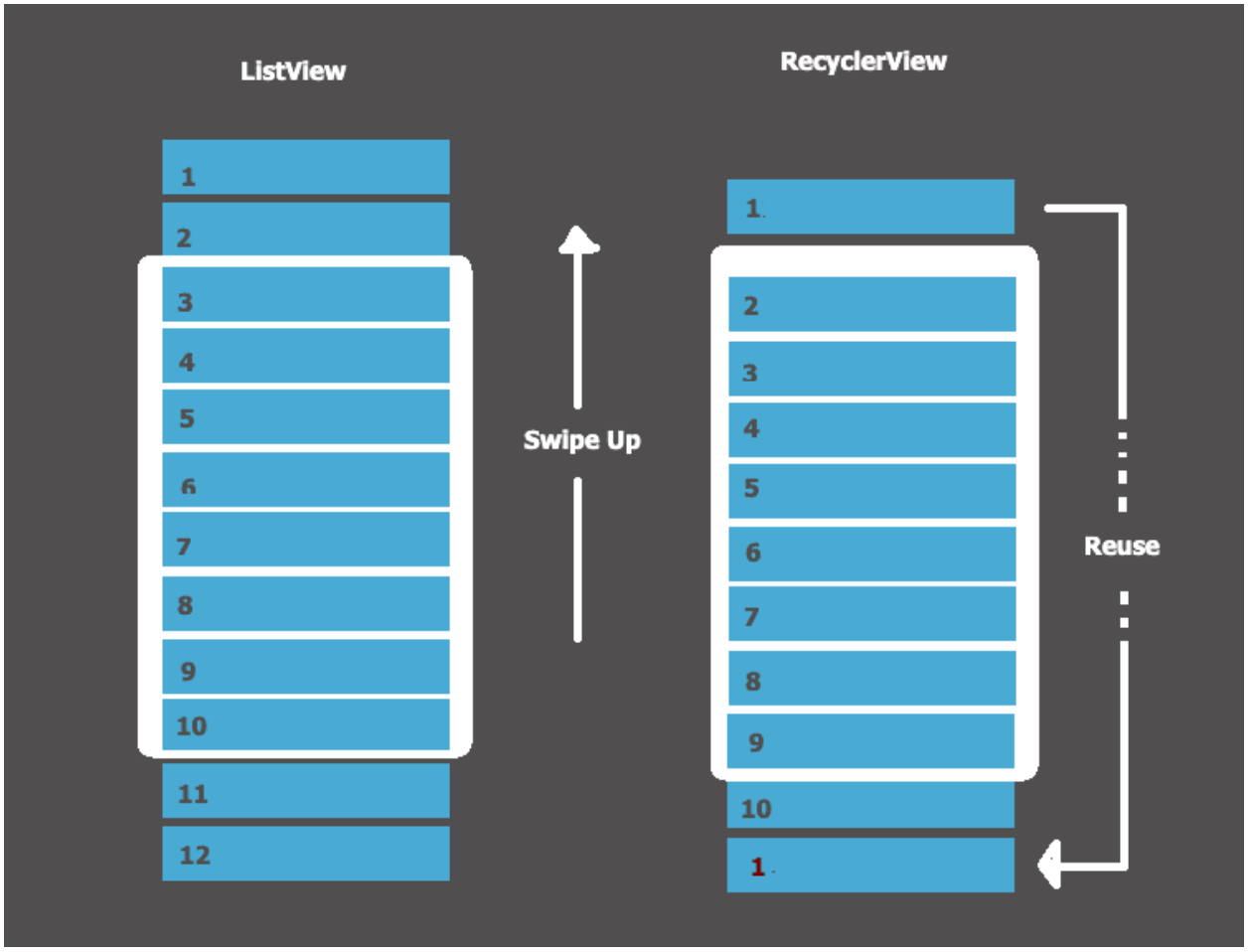


Figure 3 RecyclerView

- **Some advantages of using RecyclerView:**
  - Performance on Loading
  - View Holder Pattern
  - Layout Manager
  - Item Decoration

- **Activity VS Fragments**
  - An activity is a user interface component that is mainly used to construct a single screen of the application and represents the main focus of attention on a screen. An activity can host one or more fragments at a time.
  - Fragments are reusable components that are attached to and displayed within activities. It is basically a piece of an activity that enables more modular activity design. We can call a fragment a kind of sub-activity. It is always

hosted by an activity. It has its own layout and its own behavior, with its own life cycle callbacks.

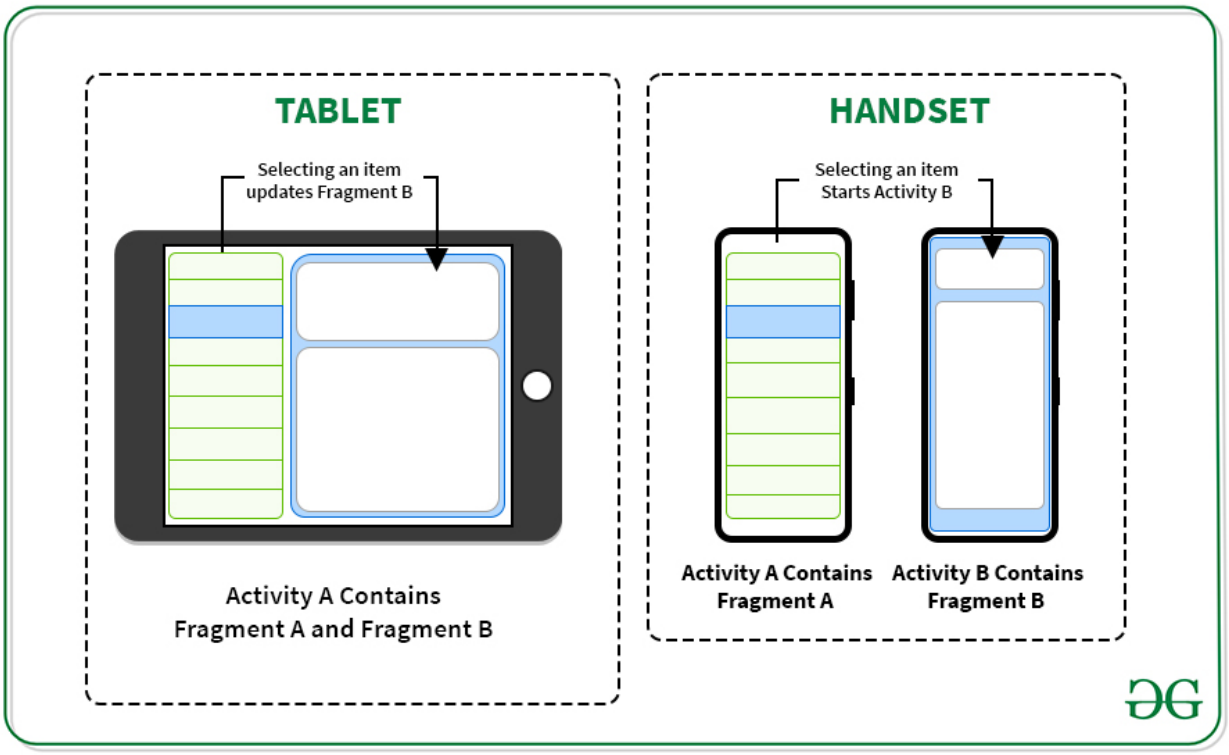
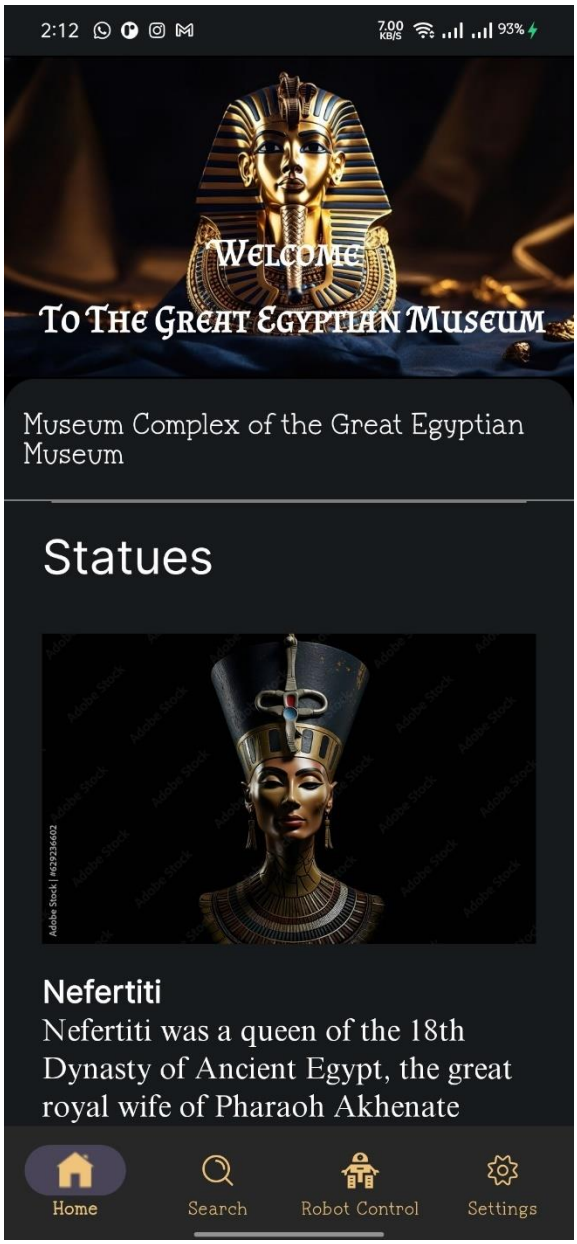


Figure 4 Activity-Fragment Layout

- **Home Page Screen**

- Dark Mode



- Light Mode

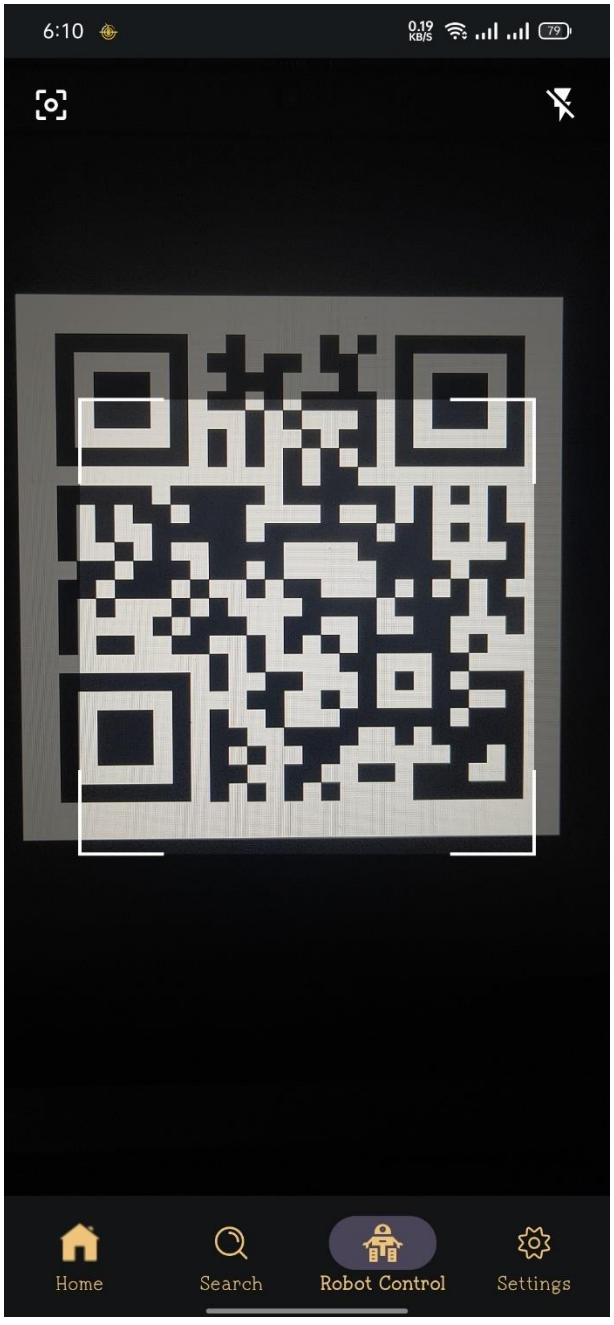


- **Artifact Brief History Screen**
  - Dark Mode
  - Light Mode

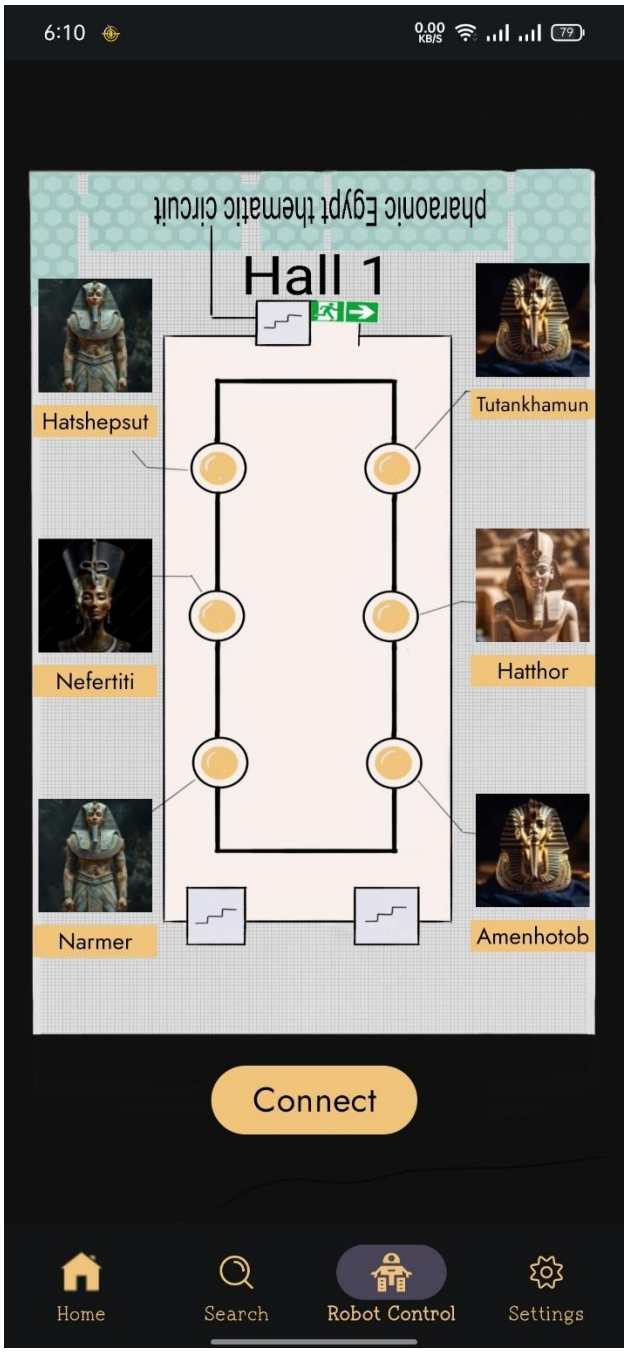




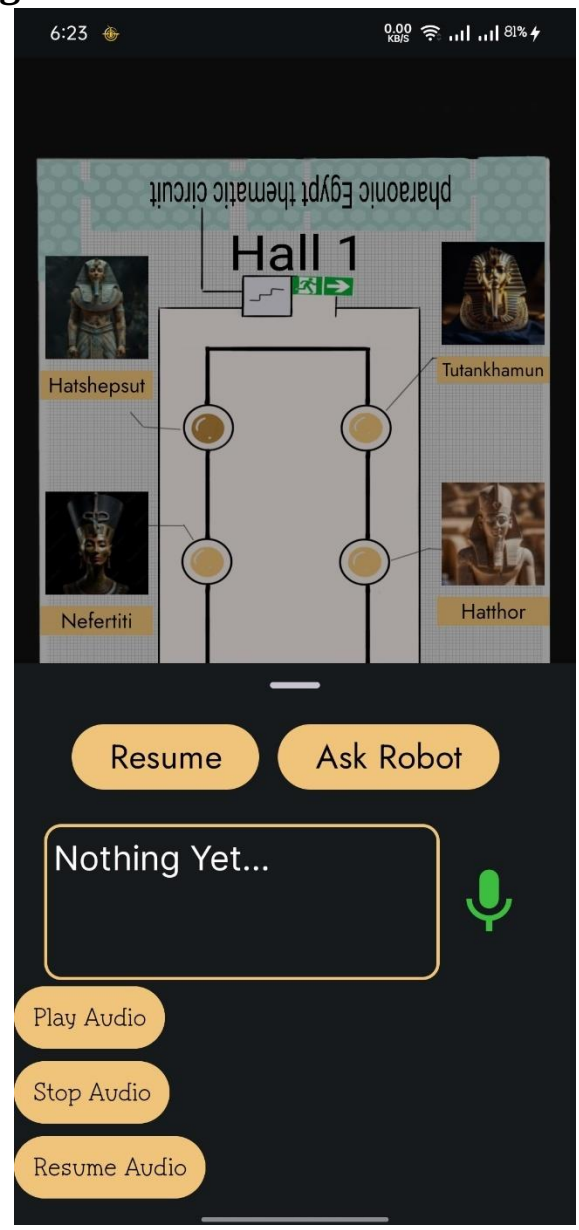
- **QR Code Scanner Screen**



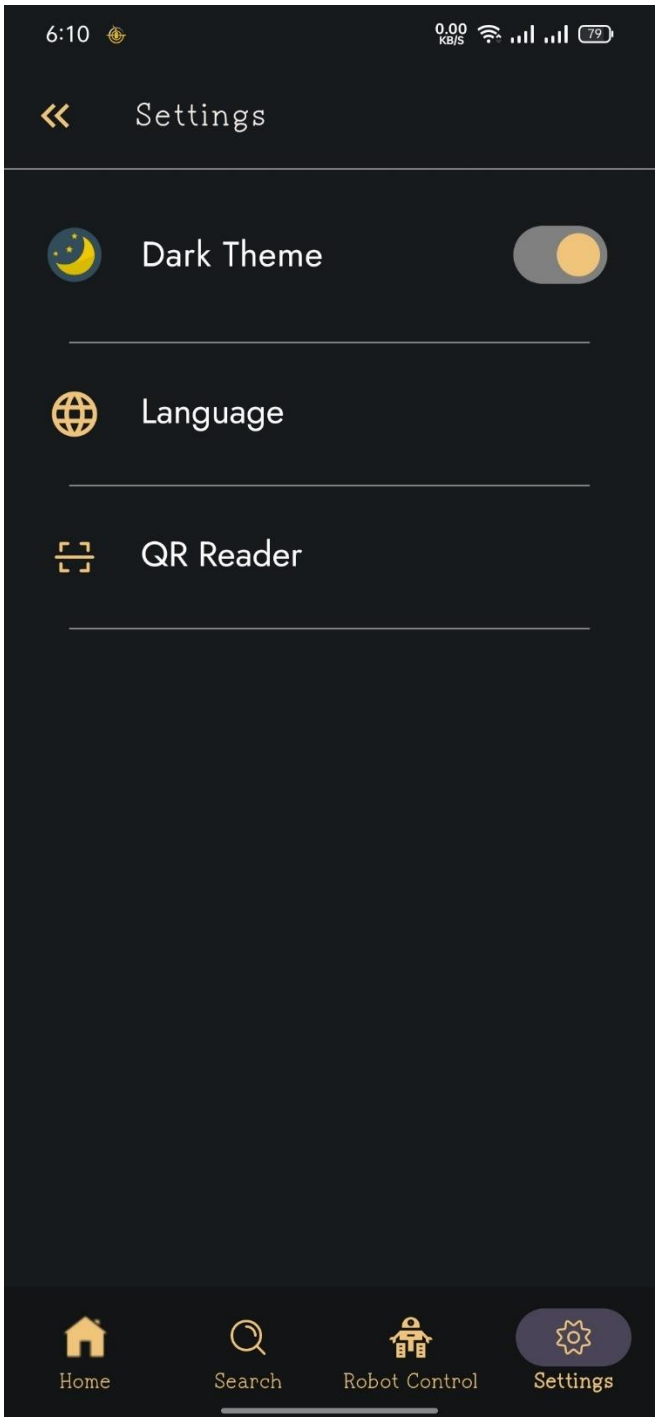
- **Robot Control Screen**



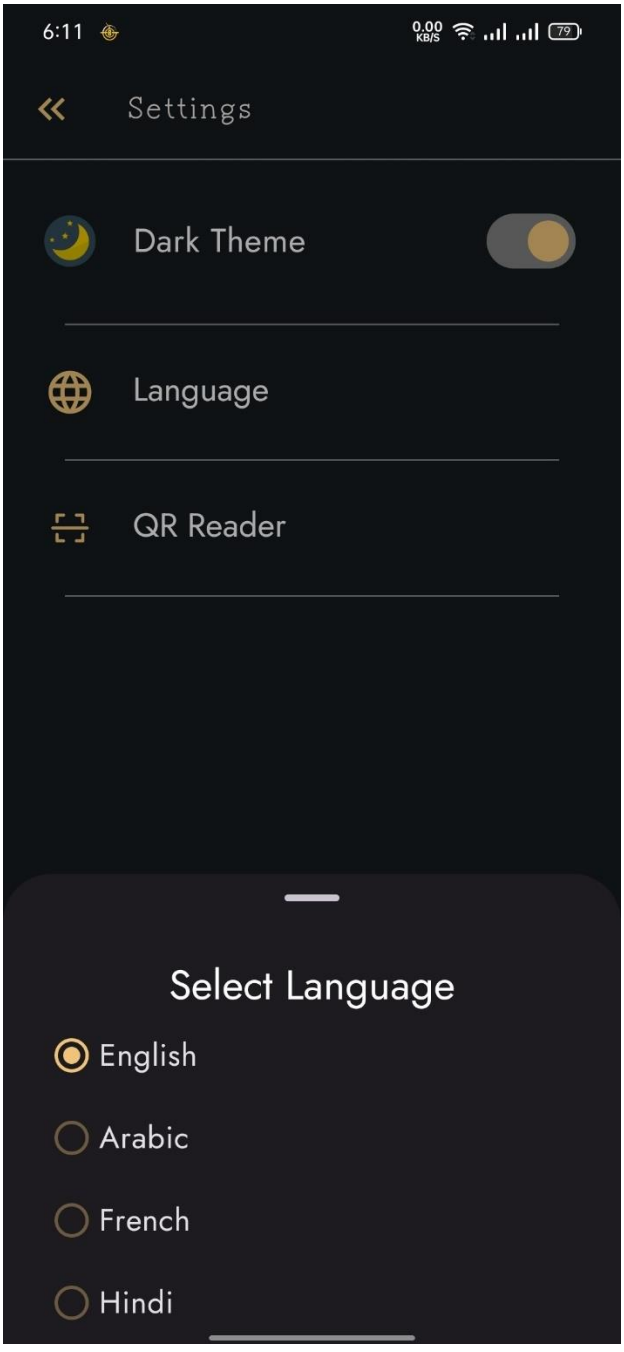
- **Bottom Sheet Fragment**



- **Settings Screen**



- **Language Change Option**



4.2 Back-end

The logic of the software is contained in the back-end design. This is the most important part of the system. It is necessary for the entire system to work. When there is a problem with the backend of the server, it leads to stopping front-end functions. The back-end includes API, AI model.

4.2.1 Architecture

We created "Application to control the robot with its connection with AI Model" by combining the following elements inthe backend:

1. **Fast API**

   Functionality: FastAPI is used to create web routes and handle HTTP requests. It offers automatic data validation, serialization, and documentation (with Swagger UI).

   Endpoint Description: The /process/ endpoint expects a POST request containing text, which it processes using various components described below.

2. **Pydantic**

   Role: Ensures that the incoming data through the API endpoint matches the expected structure (text: str), leveraging Python type hints to enforce data integrity.

3. **TensorFlow and Keras**

   Model Loading: The script loads a neural network model (model.h5) that was likely trained to classify text into various categories.
   Preprocessing: Utilizes a tokenizer and padding techniques to format raw text into a sequence of tokens that the model can interpret.

4. **NumPy**

- Usage in Prediction: After the model predicts the probabilities of different categories, NumPy is used to extract the most likely category. This is critical for mapping the model output to a human-understandable response.
- NumPy (Numerical Python) is a powerful library in Python that provides

supportfor large, multi-dimensional arrays and matrices.

Here are some common use cases and areas where NumPy is frequently utilized:

- Numerical Operations: NumPy provides a wide range of mathematical functionsand operators that can efficiently operate on arrays. It allows performing element-wise operations, such as addition, subtraction, multiplication, and division, as well as more advanced operations like trigonometric functions, exponential and logarithmic functions, and linear algebra operations.
- Data Analysis and Manipulation: NumPy is widely used in data analysis tasks. Itprovides tools for indexing, slicing, and reshaping arrays, making it easy to extract and manipulate subsets of data. NumPy arrays can be used for data cleaning, filtering, aggregation, and transformation tasks.
- Linear Algebra: NumPy offers a comprehensive set of functions for linear algebra operations, such as matrix multiplication, matrix factorization, eigenvalues, and eigenvectors computation, solving linear equations, and more. These capabilities make NumPy suitable for tasks involving linear algebra, such.

  as solving systems of equations, least squares fitting, and performing principal component analysis (PCA). Overall, NumPy is a fundamental library for scientific computing and data analysis in Python. Its ability to handle large multi-dimensional arrays, along with its mathematical functions and integrationwith other libraries, makes it a valuable tool in various domains, including dataanalysis,

machine learning, signal processing, and scientific research.



5. Pickle

Purpose: Loads pre-trained utilities like the tokenizer and label encoder, which are essential for converting text to a format suitable for the model and for interpreting the model's output.

6. JSON

Data Management: Manages a file containing potential responses mapped to model outputs, which allows for dynamic response generation based on the prediction.

4.2.3 Deep Dive into Functional Workflow

- Initialization: Components Loaded: Model, tokenizer, label encoder, and response mappings are loaded into memory, setting up the necessary infrastructure to process incoming requests.

- API Interaction:

- **Receiving Input**: Through the /process/ endpoint, the application receives raw text encapsulated in a JSON payload.
- **Data Validation**: Pydantic validates the input data structure.
- **Text Preprocessing**: Text is tokenized and padded to match the model's training format.
- **Model Prediction**: The preprocessed text is fed into the model, which returns a prediction.
- **Response Generation**: The predicted label is used to retrieve a suitable response from the pre-loaded mappings. If no appropriate response is found, a default response is provided.

4.2.4 Advanced Use Case and Enhancements

Customizable Responses: Enhance the response mechanism to include context-sensitive responses, varying not only with the label but also with specific keywords or phrases found in the input.

Continuous Learning: Implement feedback loops where the model can be fine-tuned based on user interactions and corrections to improve accuracy over time.

Scalability: Utilize Docker containers and Kubernetes for deploying the API to handle high traffic and ensure scalability and resilience.

Security Features: Implement rate limiting, authentication, and encryption to secure the API against misuse and data breaches.

4.3 Artificial intelligent (AI)

4.3.1 Python



It is a high-level programming language, which is easy to read and write. It is an open source object-oriented programming language, which offers fast development. The Python programming language is an interpretive
language with multiple uses in several functions in several languages, such as independent programs by graphical interfaces, and is used in creating the Web. Learning Python is one of the easiest tasks in programming.

4.3.1.1 The most important characteristics and advantages of thePython programming language:-

Ease of learning, as it contains easy and simple structures.

- **Python** is an open source and free language, as it is possible to distribute copies of it andmake changes to it freely.

- **Python** is a high-level language, so you don't need to review the details. It works on all major operating systems such as Linux, Macintosh, and Windows. An object-oriented language that also supports procedural programming. Learn about object-oriented languages through this article. It has multiple uses, as it is used in many fields in manyprogramming paths.

4.3.1.2 Uses of the Python language

Python has wide uses in a large number of areas of programming. For example, you can program games through the PyGame library, which enables you to program games quickly and smoothly, and this library is an ideal start for learning the basics of programming and game design, you will learn about a set of technologies and the structure of games, this does not mean that it is not suitable for making powerful games, for example Metin2 game is based on this library, in addition to programming games you can also program desktop applications. Also, to do this, there are many libraries, the most famous of

which is GTK or the QT library. On the other hand, you can program websites in the Python language, relying on a set of frameworks, the most famous of which is Django, which is a strong and solid framework, enough that large sites use it such as Bitbucket or Pinterest, What the framework adds is an environment that provides speed, organization, and protection, and this is one of the most important factors for the success of any site. The areas of using Python do not stop here, but also include a more exciting field, which is the world of hacking experiments and other areas such as artificial intelligence. Despite this, many criticize this language as being rather slow. This is due to the structure of the language itself. On the other hand, most programmers prefer it because it is beautiful and helps productivity. In a survey conducted by Stack overflow, Python ranked sixth in the Back-End category, which is a hint of the popularity ofthis language.

4.3.2 Model

- **Model Building:**

  The architecture of the AI model is designed to efficiently process text data. The choice of using TensorFlow, particularly its Keras API, facilitates the building of a sequential model—a type of model that is appropriate for stacking layers linearly.

- **Model Architecture Details:**

  Input Layer: This layer is designed to handle input sequences of text data, which are tokenized and padded to ensure uniformity in sequence length.

  Embedding Layer: Following the input, an embedding layer transforms the tokenized text into dense vectors of fixed size. This is crucial as it helps in reducing dimensionality and capturing semantic meanings of words.

  Dense Layers: Several dense layers with activation functions like ReLU are used to introduce non-linearity, aiding the model in learning complex patterns in the data.

  Output Layer: The final layer uses a softmax activation function to output a probability distribution across the various classes (responses), facilitating the model to choose the most likely response.

Each layer is meticulously configured with parameters like number of units, activation functions, and optimizers, which are pivotal in defining the model's learning behavior and its ability to generalize over unseen data.

4.3.3 Preprocessing Techniques

Text data requires extensive preprocessing before it can be fed into neural networks. The notebook details several key preprocessing steps:

Tokenization: Converting text into tokens (words or phrases) helps in breaking down the text into manageable parts for the model to understand.
Padding: Since neural networks require input data of consistent shape and size, padding is applied to ensure all text sequences are of equal length.

Label Encoding: Converting labels into numerical format is essential for training, as machine learning models inherently work with numbers. The use of Label Encoder from Scikit-learn is a typical approach for this task.
These preprocessing steps are critical in transforming raw text data into a structured format that neural networks can work with, enhancing the model's performance.

- **Label Encoding and Data Preparation:**
  The notebook elaborates on the transformation of categorical labels into a format suitable for training neural networks. This involves converting text labels into integers and then applying one-hot encoding to transform these integer labels into binary vectors. This is done because neural networks perform better with numerical input.

- **Steps Involved:**
  Integer Encoding: Every unique label is assigned a unique integer.
  One-Hot Encoding: Each integer label is converted into a binary vector with a 1 in the position corresponding to the integer and 0s elsewhere.
  This encoding not only simplifies the representation of categorical data but also aids the neural network in effectively differentiating between different categories without any inherent ordering.

4.4 Embedded System

4.4.1 Arduino Uno

Arduino UNO is a microcontroller board based on the ATmega328P. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with a AC-to-DC adapter or battery to get started. let's delve deeper into some of the aspects of the Arduino Uno:

1. **Microcontroller**: The Atmega328P microcontroller on the Arduino Uno is based on the AVR architecture. It operates at 16 MHz and has 32 KB of flash memory for storing your program. The flash memory is non-volatile, meaning it retains its contents even when power is turned off. This makes it suitable for embedded applications where the program needs to persist across power cycles.

2. **Digital I/O Pins**: The digital pins on the Arduino Uno can be used for a variety of purposes. They can be configured as inputs to read digital signals (either HIGH or LOW) from external devices such as sensors, switches, or buttons. Conversely, they can also be configured as outputs to drive digital devices like LEDs, relays, or servo motors.

3. **Analog Inputs**: The Arduino Uno's analog input pins can measure voltages between 0 and 5 volts. The resolution of the analog-to-digital converter (ADC) is 10 bits, meaning it can represent voltages in 1024 discrete steps. This provides sufficient precision for many analog sensing applications, such as reading values from temperature sensors, light sensors, or potentiometers.

4. **PWM (Pulse Width Modulation) Pins**: The Arduino Uno has six PWM-capable pins (marked with ~ symbol) on digital pins 3, 5, 6, 9, 10, and 11. PWM allows you to simulate analog output by rapidly toggling the pin between HIGH and LOW states at varying duty cycles. By adjusting the duty cycle, you can control the average voltage output, enabling you to dim LEDs, control the speed of motors, or generate audio signals.

5. **Power Pins**: The Arduino Uno can be powered via USB, or an external power supply connected to the Vin pin. The onboard voltage regulator ensures a stable 5V supply for the microcontroller and other components. Additionally, the 3.3V pin

provides a lower voltage option suitable for interfacing with certain sensors or peripherals.

6. **Reset Button**: The reset button allows you to restart your program or reinitialize the microcontroller without needing to power cycle the board. This can be useful for debugging purposes or for initiating a new program upload.

7. **Crystal Oscillator**: The 16 MHz crystal oscillator provides the clock signal for the microcontroller, ensuring precise timing for various operations and communications.

8. **USB Interface**: The USB interface facilitates communication between the Arduino Uno and a computer for programming and serial communication. It uses a USB-to-serial converter chip to translate data between the microcontroller's UART interface and the computer's USB port.

9. **ICSP Header**: The In-Circuit Serial Programming (ICSP) header allows for low-level programming and debugging of the microcontroller using an external programmer. This is useful for advanced users or for situations where the bootloader is corrupted.

10. **UART (Universal Asynchronous Receiver/Transmitter):** The RX and TX pins provide hardware UART communication, allowing the Arduino Uno to communicate with other serial devices such as GPS modules, Bluetooth modules, or other microcontrollers.

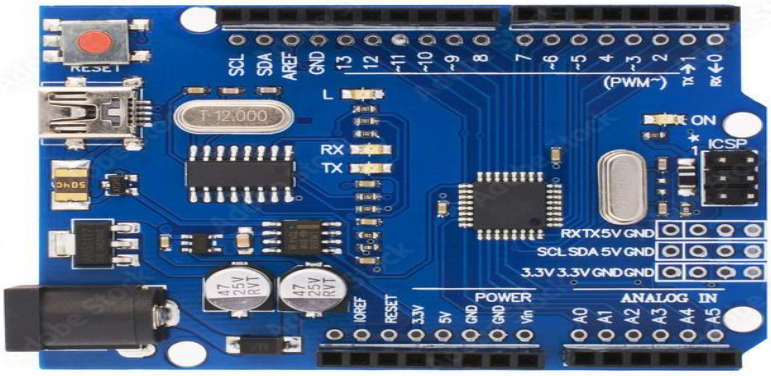


Figure 5 Arduino Uno

4.4.1 L298 Motor Driver

The L298N is a popular motor driver integrated circuit (IC) commonly used to control DC motors and stepper motors. It's widely used in robotics, mechatronics, and other projects requiring motor control. Let's break down its features and functions in detail:

1. **H-Bridge Configuration**: The L298N features two H-bridges, which are circuits commonly used to control the direction and speed of motors. Each H-bridge consists of four transistors arranged in a configuration that allows bidirectional control of a motor. By controlling the state of these transistors, the motor can be driven forward, backward, or stopped.

2. **Dual Motor Control**: With two H-bridges, the L298N can control two separate motors independently. This makes it suitable for applications such as robotics, where multiple motors need to be controlled simultaneously for movement and steering.

3. **Motor Voltage**: The L298N can handle a wide range of motor voltages, typically up to 46V. This flexibility allows it to drive a variety of motors, including low-voltage hobby motors and higher-voltage industrial motors.

4. **Current Sensing**: The L298N includes built-in current sensing functionality, which can be used to monitor the current flowing through the motors. This is useful for detecting overloads or stalls, protecting the motors from damage.

5. **Logic Inputs**: The control inputs of the L298N are compatible with standard TTL logic levels (0V to 5V), making it easy to interface with microcontrollers, Arduino boards, and other digital circuits. The control inputs determine the direction and speed of the motors.

6. **PWM (Pulse Width Modulation) Control**: The L298N supports PWM control for speed regulation. By applying a PWM signal to the enable pins of the H-bridges, you can vary the average voltage applied to the motor, thereby controlling its speed.

7. **Built-in Diodes**: The L298N includes internal flyback diodes (also known as freewheeling diodes) connected across each of the motor terminals. These diodes provide a path for the back electromotive force (EMF) generated by the motor when it is turned off, protecting the driver and other components from

voltage spikes.

8. **Heat Sink**: Since the L298N can handle relatively high currents, it can generate significant heat during operation. To dissipate this heat and prevent overheating, it's recommended to attach a heat sink to the IC.

9. **Versatile Applications**: The L298N can be used in a wide range of applications beyond motor control. For example, it can be used as a general-purpose high-current switch for controlling other types of loads, such as solenoids or lamps.

10. **Package Options**: The L298N is available in various package options, including DIP (Dual Inline Package) and multiwatt, making it easy to integrate into different types of circuits and projects.

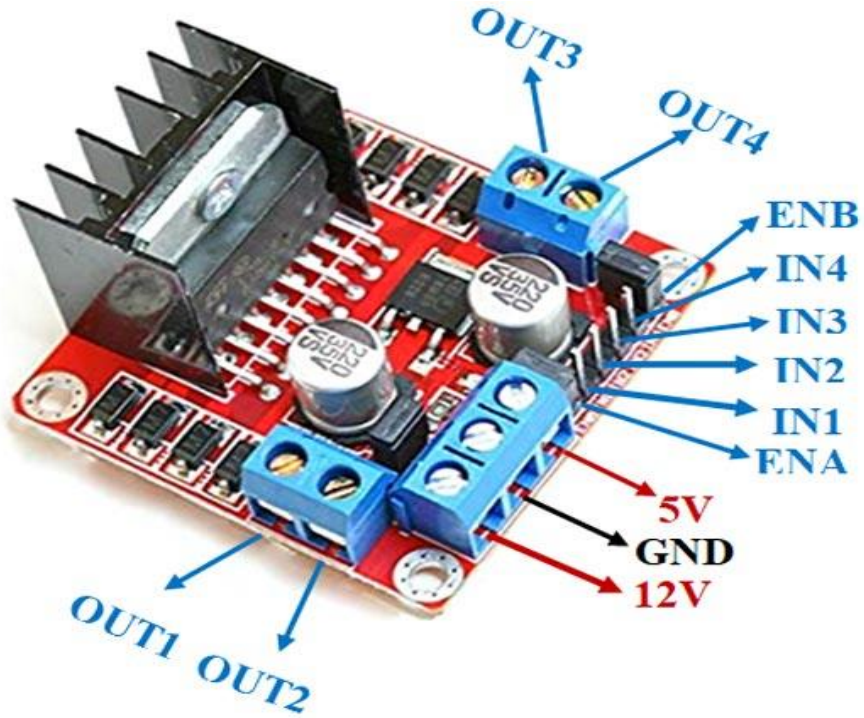


Figure 6 L298 Motor Drive

4.4.2 HC-05 Module

1- **Bluetooth Version:** The HC-05 module is based on Bluetooth version 2.0 + EDR (Enhanced Data Rate). It supports the Serial Port Profile (SPP), which allows it to establish a virtual serial port connection with other Bluetooth devices.

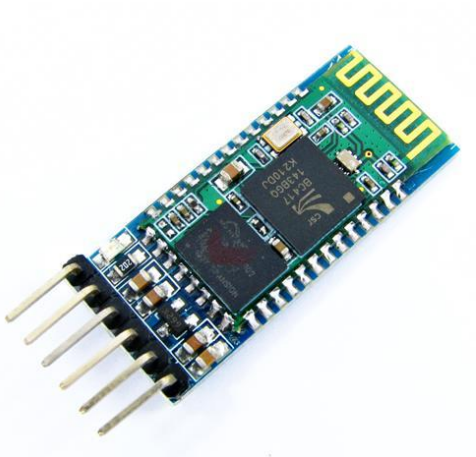


Figure 7 HC-05 Bluetooth Module

2- **Serial Communication:** The HC-05 module communicates with microcontrollers or other devices using serial communication (UART). It has RX (Receive) and TX (Transmit) pins for connecting to the serial interface of the host device. This allows for easy integration with popular microcontrollers like Arduino, Raspberry Pi, and other embedded systems.

3- **Master/Slave Mode:** The HC-05 module can operate in either Master mode or Slave mode, depending on its configuration. In Master mode, it can actively initiate connections with other Bluetooth devices (e.g., smartphones or tablets). In Slave mode, it passively waits for incoming connections from other Bluetooth devices.

4- **AT Commands:** The HC-05 module can be configured and controlled using AT commands sent over the serial interface. These commands allow you to set parameters such as the Bluetooth device name, pairing code, operating mode (Master or Slave), baud rate, and more. This flexibility makes it easy to customize the behavior of the module to suit your specific application requirements.

4-**Pairing and Security:** The HC-05 module supports pairing with other Bluetooth devices to establish a secure connection. Pairing typically involves entering a passkey (also known as a PIN) on both the HC-05 module and the other Bluetooth device. Once paired, the devices can communicate securely over the Bluetooth link.

5-**Range:** The effective range of the HC-05 module depends on factors such as the transmit power, antenna design, and environmental conditions. In typical applications, it can achieve ranges of up to 10 meters (33 feet) in open spaces. However, this range can be extended with the use of external antennas or signal amplifiers.

4.4.3 IR Sensor

An infrared (IR) sensor is a device that detects infrared radiation emitted or reflected by objects in its field of view. These sensors are commonly used in various applications, including proximity sensing, object detection, motion detection, and remote-control systems. Let's delve into its features and functions in detail:

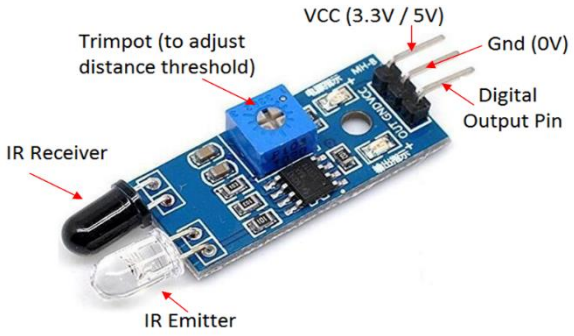


Figure 8 IR Sensor

1. **Principle of Operation**: IR sensors work based on the principle of detecting infrared radiation. All objects with a temperature above absolute zero emit thermal radiation in the form of infrared light. IR sensors utilize specialized components, such as IR photodiodes or IR receivers, to detect this radiation.

2. **Detection Range**: The detection range of an IR sensor depends on factors such as its sensitivity, the intensity of the infrared radiation emitted or reflected by the objects, and the sensor's design. Typically, IR sensors have a range of a few centimeters to several meters.

3. **Types of IR Sensors**:
   - Passive IR (PIR) Sensors: PIR sensors detect changes in infrared radiation caused by the movement of objects within their detection range. They are commonly used in motion detection applications, such as security systems and automatic lighting.
   - Active IR Sensors: Active IR sensors emit infrared radiation and measure the intensity of the reflected signal. They are used in proximity sensing and object detection applications. Examples include infrared proximity sensors and reflective IR sensors.
   - IR Receivers: IR receivers are used to detect modulated infrared signals from remote controls or other transmitting devices. They demodulate the received signal and output a corresponding electrical signal, allowing devices to respond to remote commands.
4. **Spectral Range**: IR sensors operate in the infrared portion of the electromagnetic spectrum. The spectral range of an IR sensor determines the types of objects it can detect. For example, near-infrared (NIR) sensors are sensitive to wavelengths closer to the visible spectrum and are used in applications such as night vision and optical

communication.

5. **Response Time**: The response time of an IR sensor refers to the time it takes to detect changes in infrared radiation and output a corresponding electrical signal. Faster response times are desirable in applications requiring rapid detection of moving objects or changes in the environment.

6. **Operating Conditions**: IR sensors may have specific operating conditions, such as temperature and humidity ranges, that affect their performance. It's essential to consider these factors when selecting and deploying IR sensors in different environments and applications.

7. **Signal Processing**: In some applications, IR sensors may require additional signal processing to filter out noise, compensate for ambient conditions, or extract relevant information from the detected signals. Signal processing techniques such as filtering, amplification, and digital signal processing (DSP) may be employed to enhance the sensor's performance.

8. **Applications**: IR sensors find widespread use in various industries and applications, including:
- Proximity sensing and object detection in robotics and automation.
- Motion detection in security systems, lighting control, and smart home devices.
- Object tracking and position sensing in industrial machinery and conveyor systems.
- Remote control systems for consumer electronics, including TVs, DVD players, and air conditioners.
- Environmental monitoring, such as temperature sensing and gas detection.

4.4.4 DC Gear Motor

DC gear motors are electromechanical devices that convert electrical energy into mechanical motion. They consist of a DC motor coupled with a gearbox, which reduces the motor's speed and increases its torque output. This combination of motor and gearbox provides precise control over speed and torque, making DC gear motors ideal for various applications requiring motion control. Let's explore their features and functions in detail:



Figure 9 DC Gear Motor

1. DC Motor: At the heart of a DC gear motor is a DC (direct current) motor, which converts electrical energy into rotational motion. DC motors come in various types, including brushed and brushless motors. Brushed DC motors are commonly used in gear motors due to their simplicity, cost-effectiveness, and ease of control.

2. Gearbox: The gearbox is a mechanical component that consists of a set of gears arranged to reduce the speed of the motor's output shaft while increasing its torque. Gearboxes can have different gear ratios, which determine the relationship between the speed of the motor shaft and the speed of the output shaft. A higher gear ratio results in lower output speed and higher torque, while a lower gear ratio produces higher output speed and lower torque.

3. Gear Types: Gearboxes in DC gear motors typically use various types of gears, including spur gears, planetary gears, and worm gears. Each type offers different trade-offs in terms of efficiency, torque output, backlash, and compactness. Planetary gear systems are commonly used in DC gear motors due to their high torque density, smooth operation, and compact size.

4. Torque: Torque is the rotational force produced by the motor's shaft. By using a gearbox, DC gear motors can achieve higher torque output compared to a standalone DC motor operating at the same speed. This increased torque is beneficial for driving heavy loads or overcoming resistance in mechanical systems.

5. Speed Control: DC gear motors allow precise control over speed by adjusting the voltage applied to the motor. Lowering the voltage reduces the motor's speed, while increasing the voltage speeds it up. Additionally, some DC gear motors may incorporate feedback mechanisms, such as encoders or tachometers, to provide

closed-loop speed control for more accurate and consistent performance.

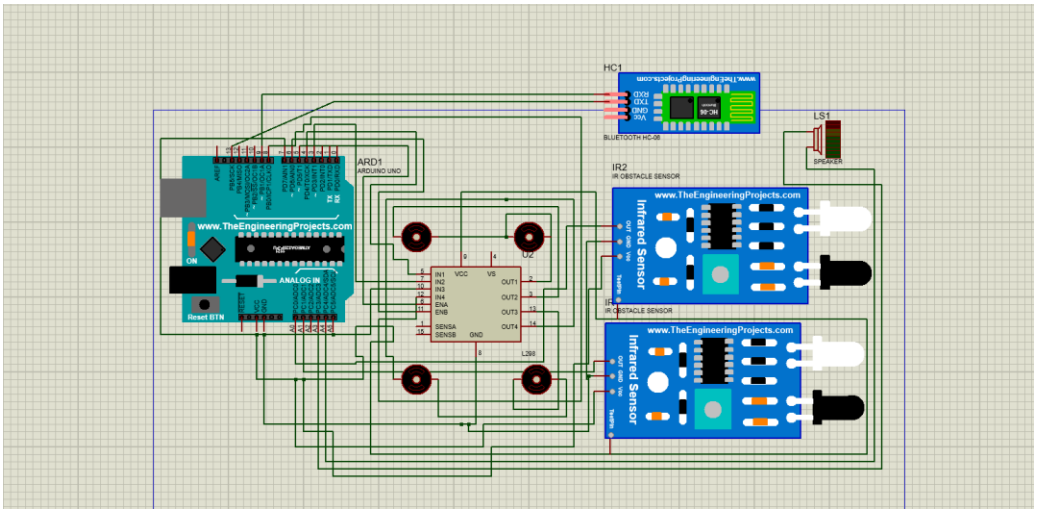
6. Direction Control: DC gear motors can change the direction of rotation by reversing the polarity of the applied voltage or by using additional control circuitry, such as an H-bridge motor driver. This bidirectional control capability makes DC gear motors suitable for applications requiring forward and reverse motion, such as robotics, actuators, and conveyor systems.

7. Efficiency: Gearboxes in DC gear motors introduce mechanical losses due to friction, which can affect their overall efficiency. Selecting the appropriate gearbox design and gear materials can help minimize these losses and improve the motor's efficiency, resulting in better performance and energy savings.

8. Applications: DC gear motors find widespread use in a variety of applications, including:

- Robotics and automation: for driving wheels, joints, and manipulators.
- Automotive: for power windows, door locks, and seat adjustments.
- Industrial machinery: for conveyors, lifts, and actuators.
- Home appliances: for fans, pumps, and electric locks.
- Medical devices: for pumps, ventilators, and mobility aids.

4.4.5 Hardware Connections



4.4.6 Speaker Module

1. **Sound Production:**
   - Sound is produced by creating vibrations in the air. These vibrations are typically generated by the diaphragm of a speaker moving back and forth.
2. **Frequency and Pitch:**
   - The frequency of the vibrations determines the pitch of the sound. Higher frequencies result in higher-pitched sounds, while lower frequencies produce lower-pitched sounds. Frequency is measured in Hertz (Hz).
3. **Amplitude and Volume:**
   - The amplitude of the sound wave determines its volume or loudness. Higher amplitudes result in louder sounds.
4. **PWM (Pulse Width Modulation):**
   - Arduino can generate sound using PWM. PWM is a method where the width of the digital pulses is varied to create different audio signals. By rapidly switching a pin between high and low states, the Arduino can approximate varying levels of voltage to drive a speaker.



5. Chapter Five

5.1 Implementation

5.1.1 Mobile Application

- **Implementing HomeFragment with ViewModel**

```
class HomeFragment : BaseFragment() {

    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding

    override val _viewModel: HomeViewModel by inject()
    override fun startAnimation() {...}

    @SuppressLint("ClickableViewAccessibility")
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentHomeBinding.inflate(inflater)

        binding!!.homeFeedRecyclerView.layoutManager = LinearLayoutManager(requireActivity())

        GlobalScope.launch { this: CoroutineScope
            _viewModel.getAllArtifacts()
        }

        val adapter = HomeAdapter(HomeItemClickListener { artifactModel ->
            _viewModel.onArtifactItemClicked(artifactModel)
        })
        binding!!.homeFeedRecyclerView.adapter = adapter

        _viewModel.navigationCommand.observe(viewLifecycleOwner) { command ->
            when (command) {
                is NavigationCommand.To -> {
                    findNavController().navigate(command.directions)
                }

                else -> {}
```

- **DetailsFragment with ViewModel**

```
11
12  class DetailsFragment : BaseFragment() {
13      private var _binding: FragmentDetailsBinding? = null
14      private val binding get() = _binding
15      override val _viewModel: DetailsViewModel by inject()
16
17      override fun startAnimation() {...}
20      override fun onCreateView(
21          inflater: LayoutInflater, container: ViewGroup?,
22          savedInstanceState: Bundle?
23      ): View {
24          _binding = FragmentDetailsBinding.inflate(inflater)
25          // Inflate the layout for this fragment
26          val artifactItem = DetailsFragmentArgs.fromBundle(requireArguments()).artifactModel
27
28          binding!!.artifactIv.setImageBitmap(BitmapConverter.stringToBitmap(artifactItem.image))
29          binding!!.artifactNameTv.text = artifactItem.title
30          binding!!.artifactDescriptionTv.text = artifactItem.details
31
32          return binding!!.root
33      }
34  }
```

- **RobotControl Fragment with ViewModel**

```
13
14
15  class RobotControlFragment : BaseFragment() {
16
17      private var _binding: FragmentRobotControlBinding? = null
18      private val binding get() = _binding
19
20      override val _viewModel: RobotControlViewModel by inject()
21
22      @SuppressLint("ClickableViewAccessibility")
23      override fun onCreateView(
24          inflater: LayoutInflater, container: ViewGroup?,
25          savedInstanceState: Bundle?
26      ): View {
27          _binding = FragmentRobotControlBinding.inflate(inflater)
28          // Inflate the layout for this fragment
29
30          startAnimation()
31          _viewModel.isButtonClickable( bool: true)
32          _viewModel.showLoading( bool: true)
33
34          binding!!.viewModel = _viewModel
35
36          binding!!.btnNum1.setOnClickListener { it: View!
37              showBottomSheetLanguages()
38              _viewModel.sendMessageToBluetoothModule("1")
39              binding!!.robotControlFragmentBackground.setBackgroundResource(R.drawable.bk_btn_1)
40          }
41
42          binding!!.btnNum2.setOnClickListener { it: View!
43              _viewModel.sendMessageToBluetoothModule("2")
44              binding!!.robotControlFragmentBackground.setBackgroundResource(R.drawable.bk_btn_2)
va > com > example > musuemguide > userSection > ui > RobotControlFragment > onCreateView
```

- **BottomSheetFragment for attached to RobotControlFragment**

```
class RobotControlBottomSheetFragment : BottomSheetDialogFragment(), TextToSpeech.OnInitListener {

    val _viewModel: RobotControlViewModel by inject()

    private var isRunning = true
    private var _binding: RobotControlBottomSheetBinding? = null
    private val binding get() = _binding
    var speechRecognizer: SpeechRecognizer? = null
    private var recognizerIntent: Intent? = null

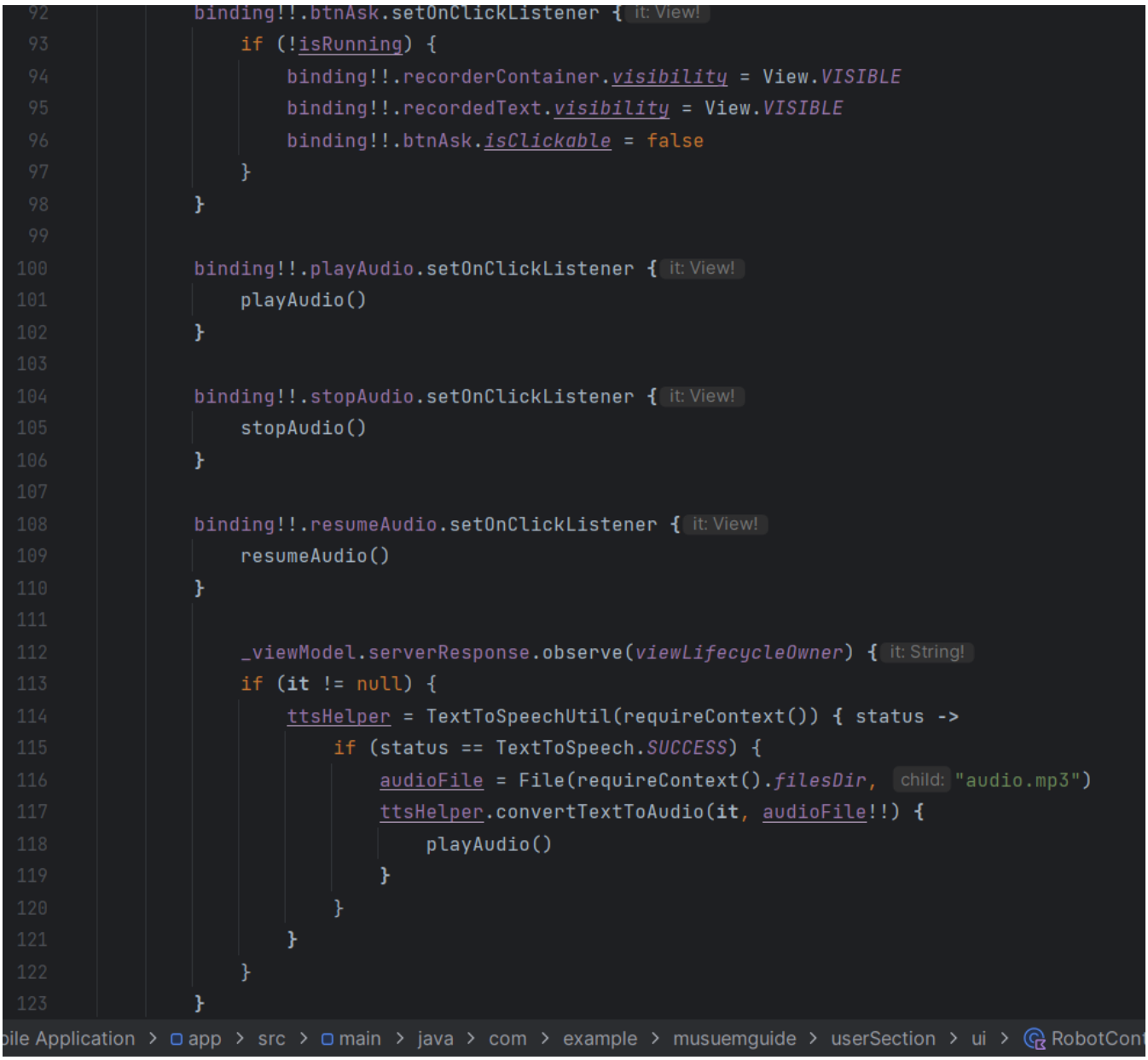
    //Text To Speech vars
    private lateinit var ttsHelper: TextToSpeechUtil
    private var mediaPlayer: MediaPlayer? = null
    private var audioFile: File? = null

    private lateinit var sharedPreferences: SharedPreferences

    override fun onInit(status: Int) {
        if (status == TextToSpeech.SUCCESS) {
            audioFile = File(requireContext().filesDir, child: "audio.mp3")
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        sharedPreferences =
            requireActivity().getSharedPreferences( name: "theme_prefs", Context.MODE_PRIVATE)
    }
```

- **RobotControlBottomSheetFragment Cont.**



```
92          binding!!.btnAsk.setOnClickListener { it: View!
93              if (!isRunning) {
94                  binding!!.recorderContainer.visibility = View.VISIBLE
95                  binding!!.recordedText.visibility = View.VISIBLE
96                  binding!!.btnAsk.isClickable = false
97              }
98          }
99
100         binding!!.playAudio.setOnClickListener { it: View!
101             playAudio()
102         }
103
104         binding!!.stopAudio.setOnClickListener { it: View!
105             stopAudio()
106         }
107
108         binding!!.resumeAudio.setOnClickListener { it: View!
109             resumeAudio()
110         }
111
112             _viewModel.serverResponse.observe(viewLifecycleOwner) { it: String!
113             if (it != null) {
114                 ttsHelper = TextToSpeechUtil(requireContext()) { status ->
115                     if (status == TextToSpeech.SUCCESS) {
116                         audioFile = File(requireContext().filesDir, child: "audio.mp3")
117                         ttsHelper.convertTextToAudio(it, audioFile!!) {
118                             playAudio()
119                         }
120                     }
121                 }
122             }
123         }
```

ile Application › app › src › main › java › com › example › musuemguide › userSection › ui › RobotCont

- **RobotControlBottomSheetFragment Cont.**
  - This code is for handling the robot controls (Stop/Go/Ask Question)

```
override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View {
    _binding = RobotControlBottomSheetBinding.inflate(inflater)

    setListeners()
    checkPermissions()
    resetSpeechRecognizer()
    setRecogniserIntent()
    ttsHelper = TextToSpeechUtil(requireContext(), listener: this)

    binding!!.btnStopRobot.text = "Stop Robot"

    binding!!.btnStopRobot.setOnClickListener { it: View!
        if (isRunning) {
            isRunning = false
            binding!!.btnStopRobot.text = "Resume"
            binding!!.btnAsk.isClickable = true
        } else {
            binding!!.btnStopRobot.text = "Stop Robot"
            isRunning = true
            binding!!.btnAsk.isClickable = false
            binding!!.recorderContainer.visibility = View.GONE
            binding!!.recordedText.visibility = View.GONE
        }
    }

    binding!!.btnAsk.setOnClickListener { it: View!
        if (!isRunning) {
```

on › app › src › main › java › com › example › musuemguide › userSection › u

- **RobotControlBottomSheetFragment Cont.**

```
125         }
126
127         private fun playAudio() {
128             if (audioFile != null && !audioFile!!.exists()) {
129                 return
130             }
131             mediaPlayer = MediaPlayer().apply { this: MediaPlayer
132                 setDataSource(audioFile!!.absolutePath)
133                 prepare()
134                 start()
135             }
136         }
137
138         private fun stopAudio() {
139             mediaPlayer?.let { it: MediaPlayer
140                 if (it.isPlaying) {
141                     it.stop()
142                     it.release()
143                     mediaPlayer = null
144                 }
145             }
146         }
147
148         private fun resumeAudio() {
149             mediaPlayer?.let { it: MediaPlayer
150                 if (!it.isPlaying) {
151                     it.start()
152                 }
153             }
154         }
155
156         private fun setListeners() {
bile Application > app > src > main > java > com > example > musuemguide > userSec
```

- **RobotControlBottomSheetFragment Cont.**

```
155
156     private fun setListeners() {
157
158         binding!!.startButton.setOnClickListener { it: View!
159             startListening()
160         }
161     }
162
163     private fun startListening() {
164         speechRecognizer!!.startListening(recognizerIntent)
165         binding!!.progressBar1.visibility = View.VISIBLE
166     }
167
168     private fun checkPermissions() {
169         val permissionCheck =
170             ContextCompat.checkSelfPermission(requireContext(), Manifest.per
171         if (permissionCheck != PackageManager.PERMISSION_GRANTED) {
172             ActivityCompat.requestPermissions(
173                 requireActivity(),
174                 arrayOf(Manifest.permission.RECORD_AUDIO),
175                 PERMISSIONS_REQUEST_RECORD_AUDIO
176             )
177             return
178         }
179     }
180
181     override fun onRequestPermissionsResult(
182         requestCode: Int,
183         permissions: Array<String>, grantResults: IntArray
184     ) {
185         super.onRequestPermissionsResult(requestCode, permissions, grantResu
186         if (requestCode == PERMISSIONS_REQUEST_RECORD_AUDIO) {
```

- **RobotControlBottomSheetFragment Cont.**
  - Handling the Microphone Permission Request.

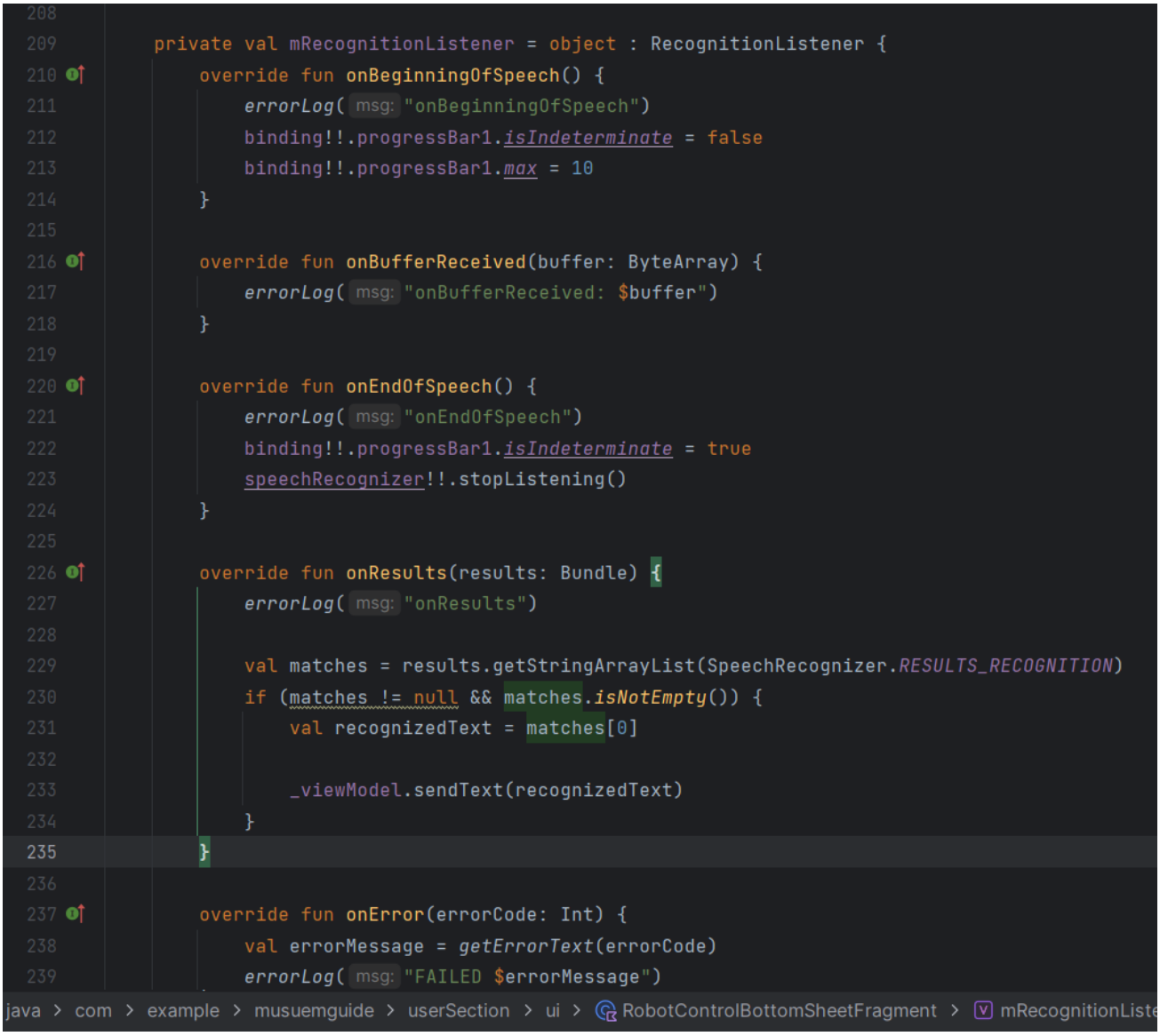
```
    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<String>, grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults)
        if (requestCode == PERMISSIONS_REQUEST_RECORD_AUDIO) {
            if (grantResults.isNotEmpty() && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                startListening()
            } else {
                //_viewModel.showToast.postValue("Permission Denied!")
            }
        }
    }

    private fun resetSpeechRecognizer() {
        if (speechRecognizer != null) speechRecognizer!!.destroy()
        speechRecognizer = SpeechRecognizer.createSpeechRecognizer(requireContext())
        errorLog(
            msg: "isRecognitionAvailable: " + SpeechRecognizer.isRecognitionAvailable(requireContext())
        )


        if (SpeechRecognizer.isRecognitionAvailable(this.requireContext())) {
            //_viewModel.showSnackBarTxt("Recognition Available")
            speechRecognizer!!.setRecognitionListener(mRecognitionListener)
        }
    }

    private val mRecognitionListener = object : RecognitionListener {
        override fun onBeginningOfSpeech() {
```

- **RobotControlBottomSheetFragment Cont.**
  - Speech Recognition
  - This code takes the user voice as input and converts it to text to send it to the server for ai interaction.



```
208
209     private val mRecognitionListener = object : RecognitionListener {
210         override fun onBeginningOfSpeech() {
211             errorLog( msg: "onBeginningOfSpeech")
212             binding!!.progressBar1.isIndeterminate = false
213             binding!!.progressBar1.max = 10
214         }
215
216         override fun onBufferReceived(buffer: ByteArray) {
217             errorLog( msg: "onBufferReceived: $buffer")
218         }
219
220         override fun onEndOfSpeech() {
221             errorLog( msg: "onEndOfSpeech")
222             binding!!.progressBar1.isIndeterminate = true
223             speechRecognizer!!.stopListening()
224         }
225
226         override fun onResults(results: Bundle) {
227             errorLog( msg: "onResults")
228
229             val matches = results.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
230             if (matches != null && matches.isNotEmpty()) {
231                 val recognizedText = matches[0]
232
233                 _viewModel.sendText(recognizedText)
234             }
235         }
236
237         override fun onError(errorCode: Int) {
238             val errorMessage = getErrorText(errorCode)
239             errorLog( msg: "FAILED $errorMessage")
```

java › com › example › musuemguide › userSection › ui › RobotControlBottomSheetFragment › mRecognitionListe

- **RobotControlBottomSheetFragment Cont.**

```
    }

    private fun setRecogniserIntent() {
        val preferredLanguage = LanguageManager.loadLanguagePreference(requireContext())
        recognizerIntent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH)
        recognizerIntent!!.putExtra(
            RecognizerIntent.EXTRA_LANGUAGE_PREFERENCE,
            preferredLanguage
        )
        recognizerIntent!!.putExtra(
            RecognizerIntent.EXTRA_LANGUAGE,
            preferredLanguage
        )
        recognizerIntent!!.putExtra(
            RecognizerIntent.EXTRA_LANGUAGE_MODEL,
            RecognizerIntent.LANGUAGE_MODEL_FREE_FORM
        )
        recognizerIntent!!.putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, RESULTS_LIMIT)
    }

    override fun onDestroy() {
        super.onDestroy()
        speechRecognizer!!.destroy()
        mediaPlayer?.release()
        ttsHelper.shutDown()
        //ttsUtil.shutdown()
    }

    override fun onResume() {
        errorLog( msg: "resume")
        super.onResume()
        resetSpeechRecognizer()
```

- **RobotControlBottomSheetFragment Cont.**

```
289
290     override fun onResume() {
291         errorLog( msg: "resume")
292         super.onResume()
293         resetSpeechRecognizer()
294         if (IS_CONTINUES_LISTEN) {
295             startListening()
296         }
297     }
298
299     override fun onPause() {
300         errorLog( msg: "pause")
301         super.onPause()
302         speechRecognizer!!.stopListening()
303     }
304
305     override fun onStop() {
306         errorLog( msg: "stop")
307         super.onStop()
308         if (speechRecognizer != null) {
309             speechRecognizer!!.destroy()
310         }
311     }
312
313 }
```

- **API Code for accessing AI model on FastAPI Server**

```
package com.example.musuemguide.serverside

import ...

interface ApiServer {
    @POST("process/")
    fun processText(@Body data: QuestionModel): Call<ResponseModel>
}
```

- **Using Retrofit to create an API Builder**

```
package com.example.musuemguide.serverside

import ...

object NetworkUtils {
    private const val BASE_URL = "http://192.168.1.3:8000/"

    val instance: ApiServer by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ApiServer::class.java)
    }

}
```

- **Repository**
  - Using MVVM Clean Architecture, Repository is one way to communicate with Local Database (Room Database) and another to communicate with Server.

```
13  class LocalRepository(
14      private val artifactDao: ArtifactDao,
15  ) : DataSource {
16
17      private val _response = MutableLiveData<String>()
18      val response: LiveData<String> = _response
19
20      override suspend fun addArtifacts(artifacts: List<ArtifactDTO>) {
21
22          artifactDao.addAllArtifacts(artifacts)
23      }
24
25      override suspend fun getAllArtifacts(): List<ArtifactDTO> {
26
27          return artifactDao.getAllArtifacts()
28      }
29
30      override suspend fun deleteAllArtifacts() {
31          artifactDao.clearDatabase()
32      }
33
34      fun sendTextAndReceiveResponse(text: String) {
35
36          val call = NetworkUtils.instance.processText(QuestionModel(text))
37          call.enqueue(object : retrofit2.Callback<ResponseModel> {
38              override fun onResponse(
39                  call: Call<ResponseModel>,
40                  response: retrofit2.Response<ResponseModel>
41              ) {
42                  if (response.isSuccessful) {
43                      _response.value = response.body()!!.response
```

a > com > example > musuemguide > localStorage > LocalRepository Gradle Build Run

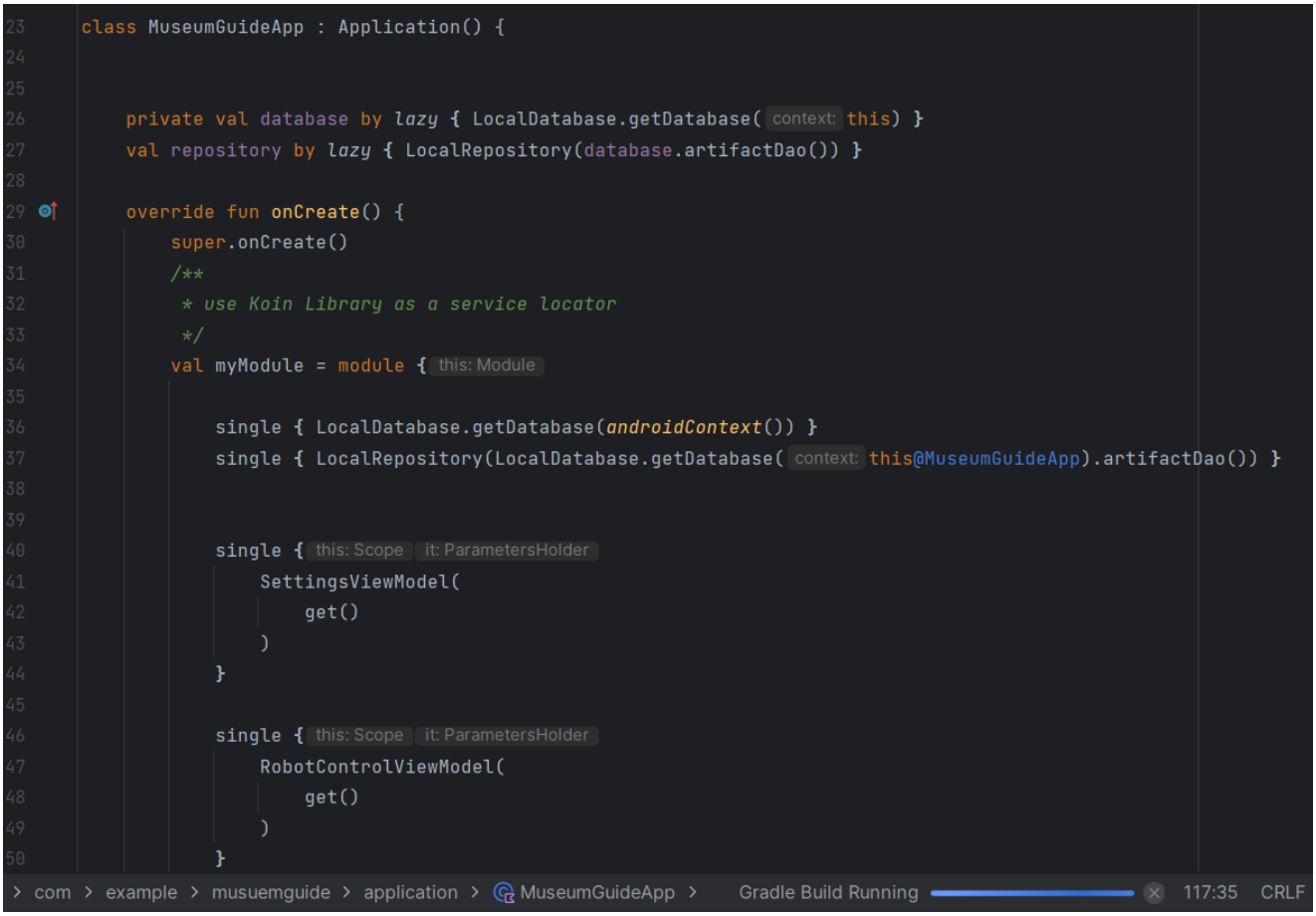
- **Repository Cont.**

```
33
34      fun sendTextAndReceiveResponse(text: String) {
35
36          val call = NetworkUtils.instance.processText(QuestionModel(text))
37          call.enqueue(object : retrofit2.Callback<ResponseModel> {
38              override fun onResponse(
39                  call: Call<ResponseModel>,
40                  response: retrofit2.Response<ResponseModel>
41              ) {
42                  if (response.isSuccessful) {
43                      _response.value = response.body()!!.response
44                  } else {
45                      _response.value = "Error: ${response.code()}"
46                  }
47              }
48
49              override fun onFailure(call: Call<ResponseModel>, t: Throwable) {
50                  _response.value = "Error: ${t.message}"
51              }
52          })
53      }
54  }
```

- **Creating Room Database**

```
8
9   private const val DATABASE_NAME = "museum_guide_database"
10
11  @Database(entities = [ArtifactDTO::class], version = 1, exportSchema = false)
12  abstract class LocalDatabase : RoomDatabase() {
13      abstract fun artifactDao(): ArtifactDao
14
15      companion object {
16          @Volatile
17          private var INSTANCE: LocalDatabase? = null
18
19          fun getDatabase(
20              context: Context
21          ): LocalDatabase {
22              // if the INSTANCE is not null, then return it,
23              // if it is, then create the database
24              return INSTANCE ?: synchronized( lock: this) {
25                  val instance = Room.databaseBuilder(
26                      context.applicationContext,
27                      LocalDatabase::class.java,
28                      DATABASE_NAME
29                  ).build()
30
31                  INSTANCE = instance
32                  // return instance
33                  instance ^synchronized
34              }
35          }
```

- **Koin**
  - To Achieve a Clean Code, easier Testing, and Efficient Performance, Koin is a Great Choice for Dependency Injection by making Singlton ModelView Classes To the whole Application.



- **Koin Cont.**

```
45
46              single { this: Scope  it: ParametersHolder
47                  RobotControlViewModel(
48                      get()
49                  )
50              }
51
52              single { this: Scope  it: ParametersHolder
53                  HomeViewModel(
54                      get()
55                  )
56              }
57
58              single { this: Scope  it: ParametersHolder
59                  DetailsViewModel(
60                      get()
61                  )
62              }
63          }
64
65          startKoin { this: KoinApplication
66              androidContext(this@MuseumGuideApp)
67              modules(listOf(myModule))
68          }
69
```

```
115         )
116         CoroutineScope(Dispatchers.IO).launch { this: CoroutineScope
117             repository.deleteAllArtifacts()
118             repository.addArtifacts(artifacts.asArtifactDTO())
119
120         }
121     }
122 }
```

- **Home Adapter**
  - Making an Adapter for the RecyclerView of the Home Screen.

```
class HomeAdapter(
    private val clickListener: HomeItemClickListener,
) : ListAdapter<DataItem, RecyclerView.ViewHolder>(HomeDiffCallback()) {

    //private val adapterScope = CoroutineScope(Dispatchers.Default)

    fun addList(list: List<ArtifactModel>) {
        submitList(list.map { artifactModel ->
            if (artifactModel.type == "HEADER") {
                //Log.d("checkkkkkk, ", "10")
                DataItem.Header(
                    HeaderModel(
                        artifactModel.id,
                        artifactModel.title,
                        artifactModel.type
                    )
                ) ^map
            } else {
                //Log.d("checkkkkkk, ", "1")
                DataItem.ArtifactItem(
                    ArtifactModel(
                        id = artifactModel.id,
                        title = artifactModel.title,
                        details = artifactModel.details,
                        category = artifactModel.category,
                        image = artifactModel.image,
                        type = artifactModel.type
```

- **Home Adapter Cont.**

```
75    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): RecyclerView.ViewHolder {
76        return when (viewType) {
77            DataItem.ItemType.HEADER.ordinal -> {
78                val headerBinding = RecyclerviewHeaderBinding.inflate(
79                    LayoutInflater.from(parent.context),
80                    parent,
81                    attachToRoot: false
82                )
83                HeaderViewHolder(headerBinding)
84            }
85
86            DataItem.ItemType.BODY.ordinal -> {
87                val artifactBinding = RecyclerviewListItemsBinding.inflate(
88                    LayoutInflater.from(parent.context),
89                    parent,
90                    attachToRoot: false
91                )
92                ArtifactViewHolder(artifactBinding)
93            }
94
95            else -> throw IllegalArgumentException("Invalid view type")
96        }
97    }
```

```
75    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): RecyclerView.ViewHolder {
76        return when (viewType) {
77            DataItem.ItemType.HEADER.ordinal -> {
78                val headerBinding = RecyclerviewHeaderBinding.inflate(
79                    LayoutInflater.from(parent.context),
80                    parent,
81                    attachToRoot: false
82                )
83                HeaderViewHolder(headerBinding)
84            }
85
86            DataItem.ItemType.BODY.ordinal -> {
87                val artifactBinding = RecyclerviewListItemsBinding.inflate(
88                    LayoutInflater.from(parent.context),
89                    parent,
90                    attachToRoot: false
91                )
92                ArtifactViewHolder(artifactBinding)
93            }
94
95            else -> throw IllegalArgumentException("Invalid view type")
96        }
97    }
```

- **Home Adapter Cont.**

```
99    override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position: Int) {
100       val item = getItem(position)
101       val animator = AnimationUtils.loadAnimation(holder.itemView.context, R.anim.cardview_anim)
102       when (holder.itemViewType) {
103           DataItem.ItemType.HEADER.ordinal -> {
104               //Log.d("checkkkkkk, ", "6")
105               val headerHolder = holder as HeaderViewHolder
106
107               holder.itemView.startAnimation(animator)
108               headerHolder.bind(item as DataItem.Header)
109           }
110
111           DataItem.ItemType.BODY.ordinal -> {
112               //Log.d("checkkkkkk, ", "7")
113               // Bind artifact data
114               val artifactHolder = holder as ArtifactViewHolder
115               val artifactItem = item as DataItem.ArtifactItem
116               holder.itemView.startAnimation(animator)
117               artifactHolder.bind(clickListener, artifactItem)
118           }
119       }
120   }
121
122   override fun getItemViewType(position: Int): Int {
123       //Log.d("checkkkkkk, ", "8")
124       return getItem(position)!!.type.ordinal
125   }
126
```

- **Home Adapter Cont.**

```
class HomeItemClickListener(val clickListener: (artifactModel: ArtifactModel) -> Unit) {
    fun onClick(artifact: ArtifactModel) = clickListener(artifact)
}

class HomeDiffCallback : DiffUtil.ItemCallback<DataItem>() {
    override fun areItemsTheSame(oldItem: DataItem, newItem: DataItem): Boolean {
        return oldItem.type == newItem.type
    }

    override fun areContentsTheSame(oldItem: DataItem, newItem: DataItem): Boolean {
        return oldItem == newItem
    }
}

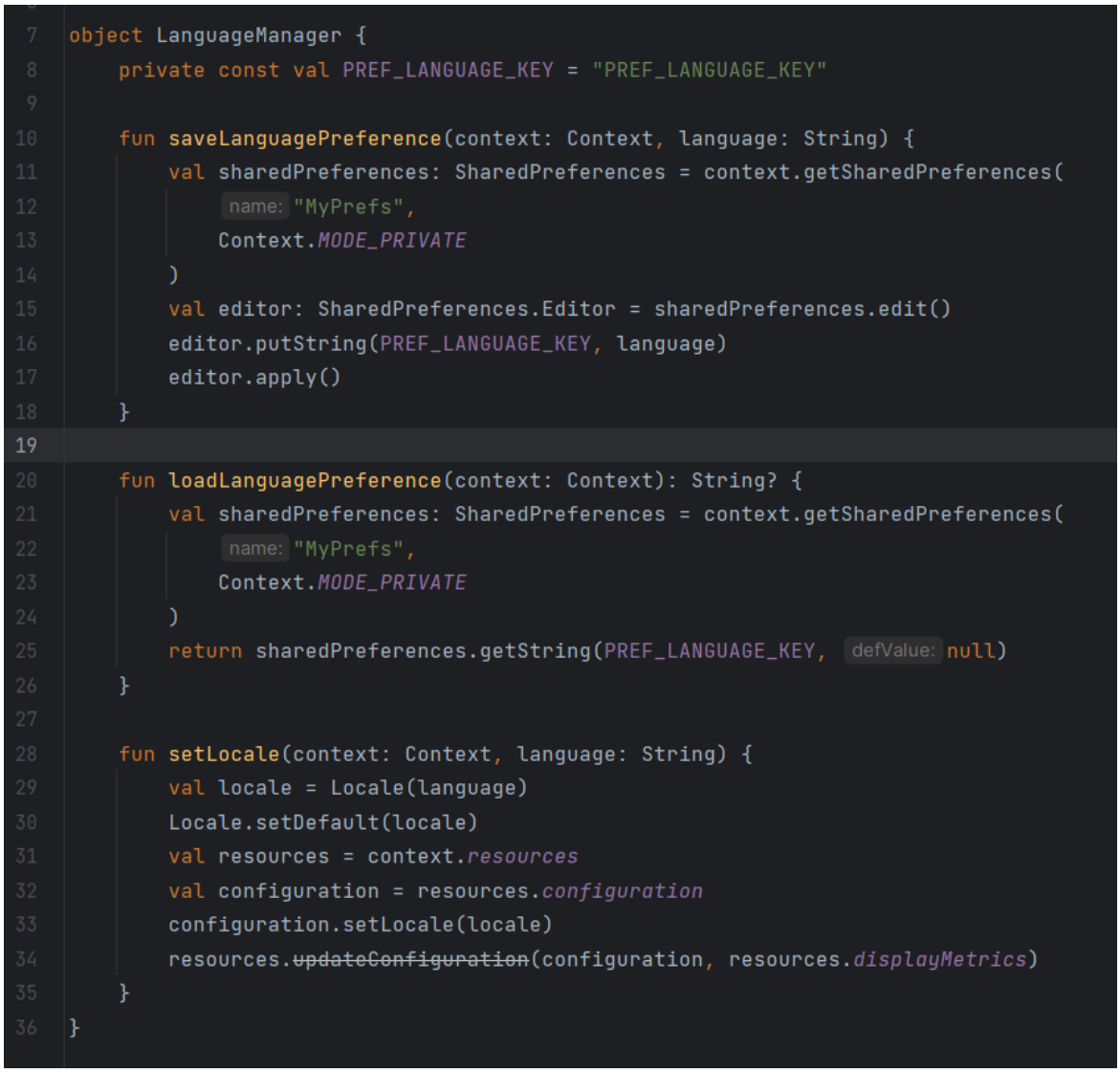
sealed class DataItem {
    data class ArtifactItem(val artifact: ArtifactModel) : DataItem() {
        override val type: ItemType = ItemType.BODY
    }

    data class Header(val headerArtifact: HeaderModel) : DataItem() {
        override val type: ItemType = ItemType.HEADER
    }

    abstract val type: ItemType

    enum class ItemType {
        HEADER,
        BODY
    }
}
```

- **Language Manager**
  - Making a Language Class to Handle User Preferred Language.



```
object LanguageManager {
    private const val PREF_LANGUAGE_KEY = "PREF_LANGUAGE_KEY"

    fun saveLanguagePreference(context: Context, language: String) {
        val sharedPreferences: SharedPreferences = context.getSharedPreferences(
            name: "MyPrefs",
            Context.MODE_PRIVATE
        )
        val editor: SharedPreferences.Editor = sharedPreferences.edit()
        editor.putString(PREF_LANGUAGE_KEY, language)
        editor.apply()
    }

    fun loadLanguagePreference(context: Context): String? {
        val sharedPreferences: SharedPreferences = context.getSharedPreferences(
            name: "MyPrefs",
            Context.MODE_PRIVATE
        )
        return sharedPreferences.getString(PREF_LANGUAGE_KEY, defValue: null)
    }

    fun setLocale(context: Context, language: String) {
        val locale = Locale(language)
        Locale.setDefault(locale)
        val resources = context.resources
        val configuration = resources.configuration
        configuration.setLocale(locale)
        resources.updateConfiguration(configuration, resources.displayMetrics)
    }
}
```

- **Speech Recognition**
  - A Helper Class to Handle Speech Recognition Operations.

SpeachRecognitionUtils.kt

```kotlin
package com.example.musuemguide.utils

import ...

const val LOG_TAG = "rum==##"
const val PERMISSIONS_REQUEST_RECORD_AUDIO = 100
const val RESULTS_LIMIT = 1

const val IS_CONTINUES_LISTEN = false

fun errorLog(msg: String?) {
    Log.e(LOG_TAG, msg!!)
}

fun getErrorText(errorCode: Int): String {
    val message: String = when (errorCode) {
        SpeechRecognizer.ERROR_AUDIO -> "Audio recording error"
        SpeechRecognizer.ERROR_CLIENT -> "Client side error"
        SpeechRecognizer.ERROR_INSUFFICIENT_PERMISSIONS -> "Insufficient permissions"
        SpeechRecognizer.ERROR_NETWORK -> "Network error"
        SpeechRecognizer.ERROR_NETWORK_TIMEOUT -> "Network timeout"
        SpeechRecognizer.ERROR_NO_MATCH -> "No match"
        SpeechRecognizer.ERROR_RECOGNIZER_BUSY -> "RecognitionService busy"
        SpeechRecognizer.ERROR_SERVER -> "error from server"
        SpeechRecognizer.ERROR_SPEECH_TIMEOUT -> "No speech input"
        SpeechRecognizer.ERROR_LANGUAGE_NOT_SUPPORTED -> "Language Not supported"
        SpeechRecognizer.ERROR_LANGUAGE_UNAVAILABLE -> "Language Unavailable"
        else -> "Didn't understand, please try again."
    }
    return message
}
```

- **Text-To-Speech**
  - A Helper Class to Handle Text-To-Speech Operations.

```
class TextToSpeechUtil(
    context: Context,
    private val listener: OnInitListener
) : OnInitListener {

    private val tts: TextToSpeech = TextToSpeech(context, listener: this)
    private var isInitialized = false

    init {
        tts.language = Locale.US
    }

    override fun onInit(status: Int) {
        if (status == TextToSpeech.SUCCESS) {
            isInitialized = true
            listener.onInit(status)
        } else {
            listener.onInit(status)
        }
    }
```

- **Text-To-Speech Cont.**

```
30
31      fun convertTextToAudio(text: String, outputFile: File, onCompletion: () -> Unit) {
32          if (isInitialized) {
33              val params = HashMap<String, String>()
34              if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP) {
35                  val utteranceId = "TextToAudio"
36                  tts.synthesizeToFile(text,  params: null, outputFile, utteranceId)
37                  tts.setOnUtteranceProgressListener(object :
38                      android.speech.tts.UtteranceProgressListener() {
39                      override fun onStart(utteranceId: String) {}
40
41                      override fun onDone(utteranceId: String) {
42                          onCompletion()
43                      }
44
45                      override fun onError(utteranceId: String) {}
46                  })
47              }
48          }
49      }
50
51      fun shutDown() {
52          tts.stop()
53          tts.shutdown()
54      }
55  }
```

- **A Shared ViewModel Across the Application ViewModel's Classes**

```
9   abstract class BaseViewModel(
10      val app: Application,
11      val localRepo: LocalRepository,
12  ) : AndroidViewModel(app) {
13
14      val navigationCommand: SingleLiveEvent<NavigationCommand> = SingleLiveEvent()
15      val showSnackBar: SingleLiveEvent<String> = SingleLiveEvent()
16      val showLoading: SingleLiveEvent<Boolean> = SingleLiveEvent()
17      val showToast: SingleLiveEvent<String> = SingleLiveEvent()
18      val buttonClickable: SingleLiveEvent<Boolean> = SingleLiveEvent()
19
20  }
21
```

- **HomeViewModel**
  - As Part MVVM Clean Architecture ViewModel is Important to Separate the UI From the Data.



```
import ...

class HomeViewModel(app: Application) :
    BaseViewModel(app, (app as MuseumGuideApp).repository) {

    private val _artifacts = MutableLiveData<List<ArtifactModel>>()
    val artifact: LiveData<List<ArtifactModel>> = _artifacts

    fun getAllArtifacts() {
        try {
            viewModelScope.launch { this: CoroutineScope
                _artifacts.value = localRepo.getAllArtifacts().asArtifactModel()
            }
        } catch (e: Exception) {
            //Log.d("checkkkkk", e.message.toString())
        }
    }

    fun onArtifactItemClicked(artifact: ArtifactModel) {
        navigationCommand.value = NavigationCommand.To(
            HomeFragmentDirections.actionHomeFragmentToDetailsFragment(artifact)
        )
    }
}
```

- **RobotControlViewModel**
  - A ViewModel to Handle the “Behind the Scene” Operations
  - At Robot Control Screen User has to Connect to the Robot Bluetooth Module.

```
42
43      @SuppressLint("MissingPermission")
44      fun connectToBluetoothModule() {
45
46
47          val bluetoothAdapter = BluetoothAdapter.getDefaultAdapter()
48          val device =
49              bluetoothAdapter.getRemoteDevice( address: "00:23:09:01:48:5D")
50
51          val uuid = UUID.fromString( name: "00001101-0000-1000-8000-00805F9B34FB") // SPP UUID
52
53          try {
54
55              bluetoothSocket = device.createInsecureRfcommSocketToServiceRecord(uuid)
56              bluetoothSocket?.connect()
57
58              if (bluetoothSocket?.isConnected == true) {
59                  showSnackBarTxt( txt: "Connected to Bluetooth module")
60                  showLoading( bool: false)
61                  isButtonClickable( bool: true)
62              } else {
63                  showSnackBarTxt( txt: "Failed to connect to Bluetooth module")
64              }
65
66          } catch (e: IOException) {
67              showSnackBarTxt( txt: "Connection error: Robot Not Found in The Area!")
68              showLoading( bool: false)
69              isButtonClickable( bool: true)
70          }
71      }
72
```

- **Bluetooth Functions**

```
73      fun sendMessageToBluetoothModule(message: String) {
74          if (bluetoothSocket?.isConnected == true) {
75              try {
76                  bluetoothSocket?.outputStream?.write(message.toByteArray())
77                  showToaster( btnName: "Message sent successfully")
78              } catch (e: IOException) {
79                  showToaster( btnName: "Failed to send message: ${e.message}")
80              }
81          } else {
82              showSnackBarTxt( txt: "Bluetooth not connected")
83          }
84      }
85
86      fun sendText(text: String){
87
88          localRepo.sendTextAndReceiveResponse(text)
89          _serverResponse.value = localRepo.response.value
90      }
91  }
```

- **Manifest File**

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3       xmlns:tools="http://schemas.android.com/tools">
4
5       <!-- Bluetooth permissions -->
6       <uses-feature
7           android:name="android.hardware.camera"
8           android:required="false" />
9
10      <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
11
12      <uses-permission android:name="android.permission.BLUETOOTH" />
13      <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
14
15      <!-- If using Bluetooth discovery -->
16      <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
17      <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
18
19      <!-- If targeting Android 12 (API level 31) or above, add this permission for Bluetooth scanning -->
20      <uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
21
22      <!-- If targeting Android 29 (API level 29) or above, add this permission for background Bluetooth -->
23      <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
24
25      <uses-permission android:name="android.permission.CAMERA" />
26
27      <!-- ***For Microphone Permission*** -->
28      <uses-permission android:name="android.permission.RECORD_AUDIO" />
29      <uses-permission android:name="android.permission.INTERNET" />
30
```

Text Merged Manifest

- **Manifest File Cont.**

```
    <queries>
        <intent>
            <action android:name="android.speech.RecognitionService" />
        </intent>
        <intent>
            <action android:name="android.intent.action.TTS_SERVICE" />
        </intent>
    </queries>

    <application
        android:name=".application.MuseumGuideApp"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="Museum Guide"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Starting"
        tools:targetApi="31"
        android:networkSecurityConfig="@xml/network_security_config">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Base.Theme.MuseumGuide">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

5.1.2 Back-End

- **API**

```python
from fastapi import FastAPI
from pydantic import BaseModel
from keras.models import load_model
from keras.preprocessing.sequence import pad_sequences
import numpy as np
import pickle
import json

app = FastAPI()

# Load the model and other necessary utilities
model = load_model('model.h5')
tokenizer = pickle.load(open("tokenizer.pkl", "rb"))
label_encoder = pickle.load(open("label_encoder.pkl", "rb"))

# Load response data
with open('g.json', 'r') as file:
    data = json.load(file)

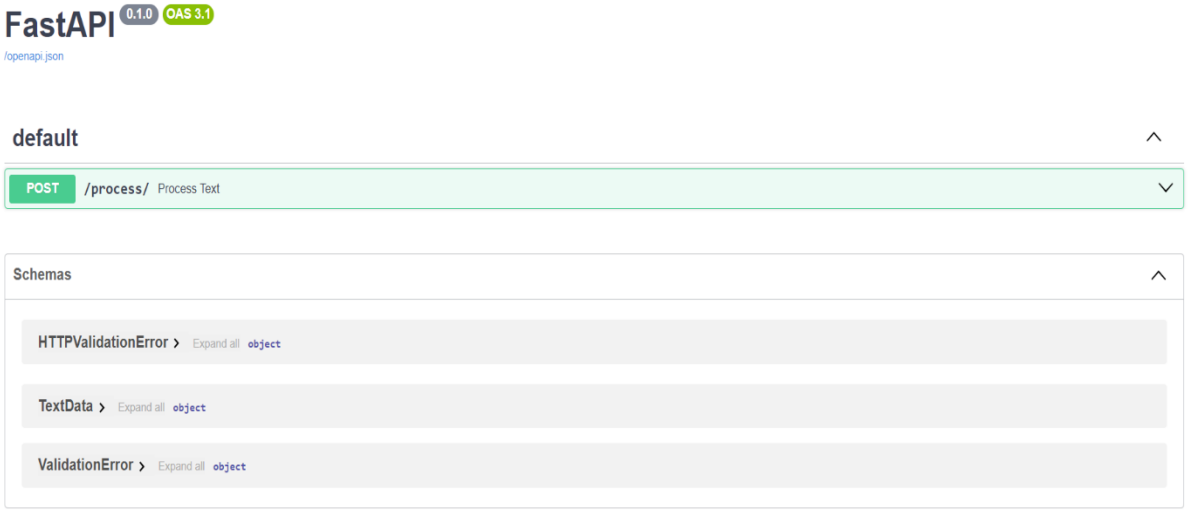
# Create a mapping from labels to responses
label_to_responses = {item['label']: item['responses'] for item in data['data']}


class TextData(BaseModel):
    text: str


def preprocess_text(text):
    # Tokenizing and padding the text to match the training format
    sequences = tokenizer.texts_to_sequences([text])
    padded = pad_sequences(sequences, maxlen=51, truncating='post')
```

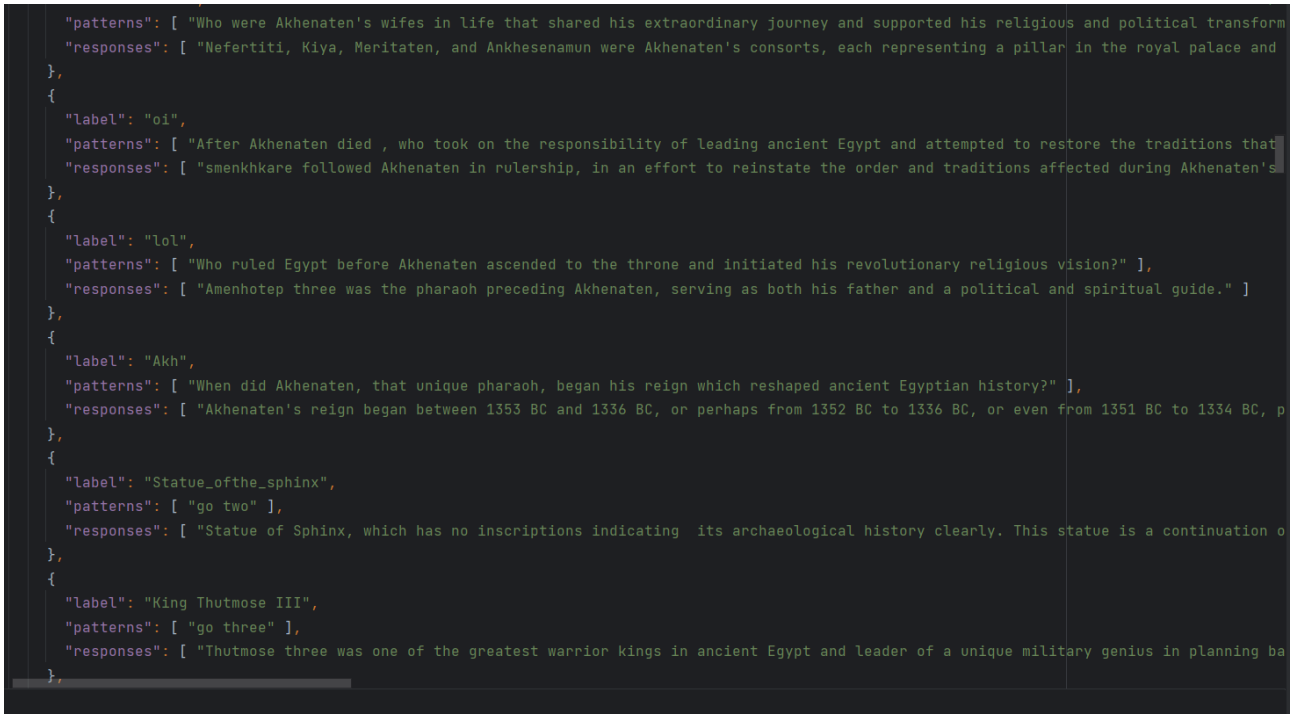
- **API Cont.**

```
27
28  def preprocess_text(text):
29      # Tokenizing and padding the text to match the training format
30      sequences = tokenizer.texts_to_sequences([text])
31      padded = pad_sequences(sequences, maxlen=51, truncating='post')
32      return padded
33
34
35  @app.post("/process/")
36  async def process_text(data: TextData):
37      # Preprocess the text
38      processed_input = preprocess_text(data.text)
39      # Predict and decode the prediction
40      raw_prediction = model.predict(processed_input)
41      prediction_index = np.argmax(raw_prediction, axis=1)
42      prediction_label = label_encoder.inverse_transform(prediction_index)
43
44      # Fetch the responses associated with the predicted label
45      responses = label_to_responses.get(prediction_label[0], ["Sorry, I can't help with that."])
46
47      # Select a random response to return
48      response_text = np.random.choice(responses)
49      return {"response": response_text}
50
```



- **AI Model**


```
  "label": "the_female_peasent",
  "patterns": [ "go one" ],
  "responses": [ "The Femal Peasent: This statue is considered one of the prominent works of the artist Mahmoud Mokhtar, in which he focused
},
{
  "label": "Akhenaten",
  "patterns": [ "how long Akhenaten was a king" ],
  "responses": [ "Akhenaten, was a king (1353◆36 BCE) of ancient Egypt of the 18th dynasty,." ]
},
{
  "label": "Akheten1",
  "patterns": [ "Who was Akhenaten?," ],
  "responses": [ "Akhenaten was an ancient Egyptian pharaoh who ruled during the 18th dynasty of the New Kingdom period. He is known for his
},
{
  "label": "Aaten1",
  "patterns": [ " Who was Akhenaten's mother? " ],
  "responses": [ "Tiye" ]
},
{
  "label": "Akhena1",
  "patterns": [ "What religious practice did Akhenaten introduce in Egypt?," ],
  "responses": [ "Atenism", "Aten" ]
},
{
  "label": "Akhten1",
  "patterns": [ "What happened to Akhenaten's monuments and statues after his death?" ],
  "responses": [ "They were dismantled, hidden, and destroyed, his name was excluded from lists of rulers" ]
```



```
    "patterns": [ "Who were Akhenaten's wifes in life that shared his extraordinary journey and supported his religious and political transform
    "responses": [ "Nefertiti, Kiya, Meritaten, and Ankhesenamun were Akhenaten's consorts, each representing a pillar in the royal palace and
  },
  {
    "label": "oi",
    "patterns": [ "After Akhenaten died , who took on the responsibility of leading ancient Egypt and attempted to restore the traditions that
    "responses": [ "smenkhkare followed Akhenaten in rulership, in an effort to reinstate the order and traditions affected during Akhenaten's
  },
  {
    "label": "lol",
    "patterns": [ "Who ruled Egypt before Akhenaten ascended to the throne and initiated his revolutionary religious vision?" ],
    "responses": [ "Amenhotep three was the pharaoh preceding Akhenaten, serving as both his father and a political and spiritual guide." ]
  },
  {
    "label": "Akh",
    "patterns": [ "When did Akhenaten, that unique pharaoh, began his reign which reshaped ancient Egyptian history?" ],
    "responses": [ "Akhenaten's reign began between 1353 BC and 1336 BC, or perhaps from 1352 BC to 1336 BC, or even from 1351 BC to 1334 BC, p
  },
  {
    "label": "Statue_ofthe_sphinx",
    "patterns": [ "go two" ],
    "responses": [ "Statue of Sphinx, which has no inscriptions indicating  its archaeological history clearly. This statue is a continuation o
  },
  {
    "label": "King Thutmose III",
    "patterns": [ "go three" ],
    "responses": [ "Thutmose three was one of the greatest warrior kings in ancient Egypt and leader of a unique military genius in planning ba
  },
```

import
json
import re
from sklearn.preprocessing import LabelEncoder
import numpy as np
import tensorflow as tf
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
with open("data1.json") as file:
  data=json.load(file)
data
training_example=[]
training_labels=[]
response =[]

for example in data['data']:
: for pattern in example['patterns']
    training_example.append(pattern)
    training_labels.append(example['label'])
  response.append(example['responses'])
print("training example ")
print(training_example)
print("training label ")
print(training_labels)
print('Response')
print(response)
len(training_labels)

```
len(set(training_labels))
training_example
import re
def clean_text(text):
 () text = text.lower
  text = re.sub(r"[^a-z0-9]+", ' ', text)
() text = text.strip
  return text
training_example = [clean_text(text) for text in training_example]
training_example = [clean_text(sentence) for sentence in training_example]
le=LabelEncoder()
le.fit(training_labels)
training_labels=le.fit_transform(training_labels)
training_labels
training_labels=to_categorical(training_labels)
training_labels
from tensorflow.keras.preprocessing.text import one_hot
sequnces=[one_hot(example,50,filters='') for example in training_example]
print(sequnces)
toknizer=Tokenizer(num_words=100)
toknizer.fit_on_texts(training_example)
sequnces=toknizer.texts_to_sequences(training_example)
print(sequnces)
padded_sequence=pad_sequences(sequnces,truncating='post',maxlen=20)
print(padded_sequence)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, GlobalAveragePooling1D

model = Sequential()
model.add(Embedding(input_dim=1000, output_dim=140, input_length=50))
model.add(GlobalAveragePooling1D())
model.add(Dense(140, activation='relu'))
model.add(Dense(140, activation='relu'))
model.add(Dense(96, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
history=model.fit(np.array(padded_sequence),training_labels,epochs=600)
import pickle as pk
pk.dump(toknizer,open("toknizer.pkl","wb"))
pk.dump(le,open("label_encoder.pkl","wb"))
model.save('model.h5')
from tensorflow.keras.models import load_model
model=load_model('model.h5')
def user_chat():
: while True
    print("User : ",end="")
()   inp=input

:   ' if inp.lower()=='quit
```

```
      break

    sent=pad_sequences(toknizer.texts_to_sequences([inp]),maxlen=20) #[] len=20

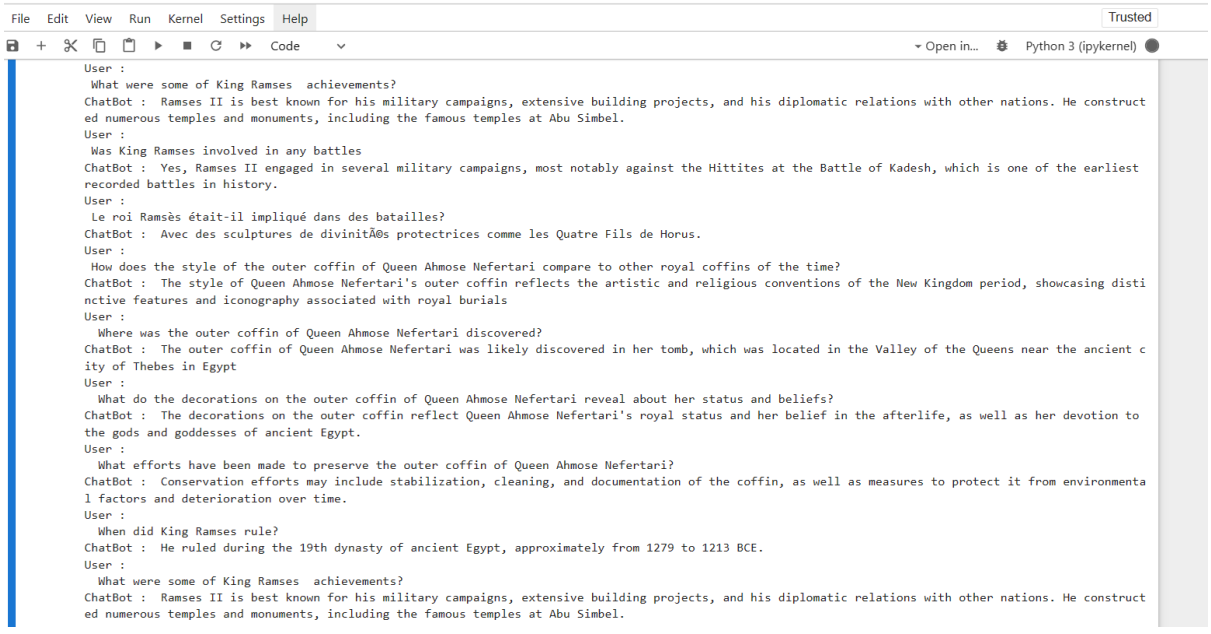
    result=np.argmax(model.predict(np.array(sent),verbose=0))
    f_res=le.inverse_transform(np.array(result).reshape(1))

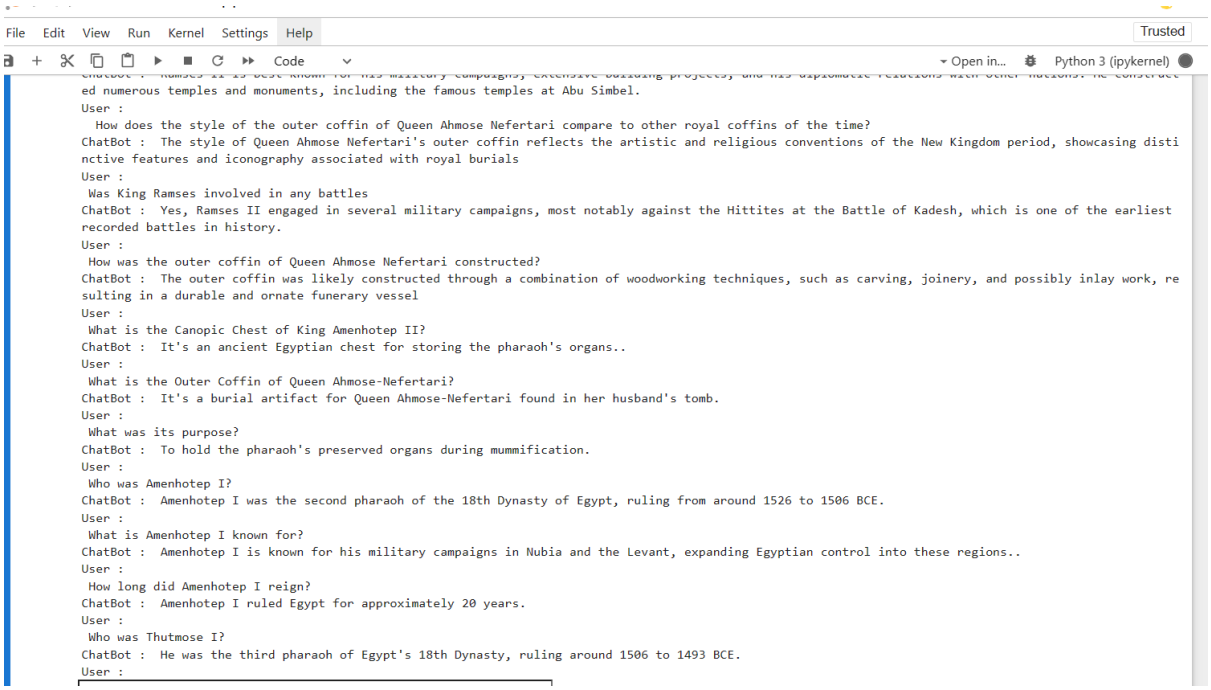

  :   for label in data['data']

 :      if label['label']==f_res

         print('ChatBot : ',np.random.choice(label['responses']))
if __name__ == "__main__":
() user_chat
```

- **AI Output**




File Edit View Run Kernel Settings Help
Trusted
Code
Open in...
Python 3 (ipykernel)
ed numerous temples and monuments, including the famous temples at Abu Simbel.
User :
How does the style of the outer coffin of Queen Ahmose Nefertari compare to other royal coffins of the time?
ChatBot : The style of Queen Ahmose Nefertari's outer coffin reflects the artistic and religious conventions of the New Kingdom period, showcasing distinctive features and iconography associated with royal burials
User :
Was King Ramses involved in any battles
ChatBot : Yes, Ramses II engaged in several military campaigns, most notably against the Hittites at the Battle of Kadesh, which is one of the earliest recorded battles in history.
User :
How was the outer coffin of Queen Ahmose Nefertari constructed?
ChatBot : The outer coffin was likely constructed through a combination of woodworking techniques, such as carving, joinery, and possibly inlay work, resulting in a durable and ornate funerary vessel
User :
What is the Canopic Chest of King Amenhotep II?
ChatBot : It's an ancient Egyptian chest for storing the pharaoh's organs..
User :
What is the Outer Coffin of Queen Ahmose-Nefertari?
ChatBot : It's a burial artifact for Queen Ahmose-Nefertari found in her husband's tomb.
User :
What was its purpose?
ChatBot : To hold the pharaoh's preserved organs during mummification.
User :
Who was Amenhotep I?
ChatBot : Amenhotep I was the second pharaoh of the 18th Dynasty of Egypt, ruling from around 1526 to 1506 BCE.
User :
What is Amenhotep I known for?
ChatBot : Amenhotep I is known for his military campaigns in Nubia and the Levant, expanding Egyptian control into these regions..
User :
How long did Amenhotep I reign?
ChatBot : Amenhotep I ruled Egypt for approximately 20 years.
User :
Who was Thutmose I?
ChatBot : He was the third pharaoh of Egypt's 18th Dynasty, ruling around 1506 to 1493 BCE.
User :

5.1.3Embedded System

- **Robot Code**

```
#include <SoftwareSerial.h>
#include <PCM.h>
//#include <Talkie.h>
//#include <TMRpcm.h>

#define SPEAKER_PIN 9
//TMRpcm tmrpcm;

//Talkie voice;

const unsigned char sample[] PROGMEM
  -13, -1, 14, 0, -20, -1, -43, -1, 9
  };

#define IR_SENSOR_RIGHT A1
#define IR_SENSOR_LEFT A0
#define MOTOR_SPEED 180
#define SPEAKER_PIN 11

//Right motor
int enableRightMotor = 5;
int rightMotorPin1 = 4;
int rightMotorPin2 = 3;

//Left motor
int enableLeftMotor = 6;
int leftMotorPin1 = 7;
int leftMotorPin2 = 8;
```

```
void loop() {

  char data = (char)bluetooth.read();
  switch (data) {
    case '1':
      getXY(0, 3);
      calc(1);
      break;
    case '2':
      getXY(0, 2);
      calc(2);
      break;
    case '3':
      getXY(0, 1);
      calc(3);
      break;
    case '4':
      getXY(1, 3);
      calc(4);
      break;
    case '5':
      getXY(1, 2);
      calc(5);
      break;
    case '6':
      getXY(1, 1);
      calc(6);
      break;
    default:
```

- **Robot Code**

```
//Push button Pin
int pbuttonPin = 11;
int buttonState = 0;

int mask = 0, mask2 = 0, mask3 = 0, mark = 0, mark2 = 0;
int x = 0, y = 0, ox = 0, oy = 0, nx = 0, ny = 0, pos = 0;  // Current position
int state;                                                  // 1=Forward, 2=Reverse, 3=Right, 4=Left;
int pxy[2], prev[2];
int rightIRSensorValue;
int leftIRSensorValue;

#define BLUETOOTH_RX 9   // Connect TX of Bluetooth to this pin
#define BLUETOOTH_TX 13  // Connect RX of Bluetooth to this pin

SoftwareSerial bluetooth(BLUETOOTH_RX, BLUETOOTH_TX);  // Create a SoftwareSerial object


void setup() {
  Serial.begin(9600);
  bluetooth.begin(9600);
  TCCR0B = TCCR0B & B11111000 | B00000010;

  //startPlayback(sample, sizeof(sample));


  // Optionally set the speaker pin
  //tmrpcm.speakerPin = SPEAKER_PIN;
```

- **Robot Mapping Code**

```
void getXY(int x, int y) {
  nx = x;
  ny = y;
}

void calc(int roPos) {
  //nx is the target X - ox is the start X
  // X is the number of moves on X-Axis
  x = nx - ox;  //New Points - Old Points
  y = ny - oy;

  if (x > 0 && y > 0) {
    exy();
    state = 3;
    orientation();
    exx();
    state = 4;
    orientation();
    ox = nx;
    oy = ny;
    stopMotors();


  } else if (x < 0 && y < 0) {
    mark2 = 1;
    getValues();
    state = 2;
    orientation();
```

```
  } else if (x < 0 && y < 0) {
    mark2 = 1;
    getValues();
    state = 2;
    orientation();
    exy1();
    state = 3;
    orientation();
    exx1();
    state = 3;
    orientation();
    ox = nx;
    oy = ny;
    mark2 = 0;
    stopMotors();


  } else if (x == 0 && y > 0) {
    getValues();
    exy();
    ox = nx;
    oy = ny;
    stopMotors();

  } else if (x == 0 && y < 0) {
    getValues();
    state = 2;
    orientation();
    exy1();
```

- **Robot Mapping Code**

```
  state = 2;
  orientation();
  exy1();
  state = 2;
  orientation();
  ox = nx;
  oy = ny;
  stopMotors();

} else if (y == 0 && x > 0) {
  getValues();
  state = 3;
  orientation();
  exx();
  state = 4;
  orientation();
  ox = nx;
  oy = ny;
  stopMotors();

} else if (y == 0 && x < 0) {
  getValues();
  state = 4;
  orientation();
  exx1();
  state = 3;
  orientation();
  ox = nx;
  oy = ny;
```

```
  orientation();
  ox = nx;
  oy = ny;
  stopMotors();

} else if (x > 0 && y < 0) {
  getValues();
  state = 3;
  orientation();
  exx();
  state = 3;
  orientation();
  mark = 1;
  exy1();
  state = 2;
  orientation();
  ox = nx;
  oy = ny;
  mark = 0;
  stopMotors();

} else if (x < 0 && y > 0) {
  getValues();
  exy();
  state = 4;
  orientation();
  mark = 1;
  exx1();
  state = 3;
```

- **Robot Mapping Code Cont.**

```
    oy = ny;
    stopMotors();
  }

  bluetooth.print("Robot is at " + String(roPos));
}



void getValues() {
  rightIRSensorValue = digitalRead(IR_SENSOR_RIGHT);
  leftIRSensorValue = digitalRead(IR_SENSOR_LEFT);
}

void compare() {

  //  WHITE = LOW ------ LEFT = HIGH

  if (leftIRSensorValue == LOW && rightIRSensorValue == LOW) {
    //moveForward();
    //Serial.println("Low is white: " + String(leftIRSensorValue));
    if (mask2 = 0) {
      inertia();
      mask2 = 1;
    }
    moveForward();
  }
  if (leftIRSensorValue == HIGH && rightIRSensorValue == LOW) {
    moveRight();
```

- **Robot Mapping Code Cont.**

```
    }
    moveForward();
  }
  if (leftIRSensorValue == HIGH && rightIRSensorValue == LOW) {
    moveRight();
  }
  if (leftIRSensorValue == LOW && rightIRSensorValue == HIGH) {
    moveLeft();
  }


  if (leftIRSensorValue == HIGH && rightIRSensorValue == HIGH && mask == 0) {
    stopMotors();

    mask = 1;
    //mask2 = 2;

    if (y > 0 && x < 0 && mark == 1 || y < 0 && x > 0 && mark == 1) {
      pos--;
    } else if (x < 0 && y < 0 && mark2 == 1) {
      pos--;
    }else if (x == 0 && y < 0 || y == 0 && x < 0) {
      pos--;
    }
    else {
      pos++;
    }

    moveForward();
```

- **Robot Mapping Code Cont.**

```
    moveForward();
    //goToStatusNumTwo(leftSensor, rightSensor);
  }

  if (leftIRSensorValue == LOW && rightIRSensorValue == LOW) {
    mask = 0;
  }
}

void inertia() {
  analogWrite(enableRightMotor, abs(250));
  analogWrite(enableLeftMotor, abs(250));

  digitalWrite(leftMotorPin1, HIGH);
  digitalWrite(leftMotorPin2, LOW);

  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);
  delay(100);
}


void orientation() {
  if (state == 2) {
    //Reverse Orientation
    moveRight180();
    stopMotors();
    state = 0;
  } else if (state == 3) {
```

- **Robot Mapping Code Cont.**

```
    stopMotors();
    state = 0;
  } else if (state == 3) {
    delay(2000);
    inertia();
    moveRight90();
    stopMotors();
    state = 0;
  } else if (state == 4) {
    delay(2000);
    inertia();
    moveLeft90();
    stopMotors();
    state = 0;
  }
}


void exy() {
loop7:
  pxy[1] = oy + pos;

  prev[1] = pxy[1];
  if (y > pos) {
    getValues();
    compare();
  } else if (y == pos) {
    delay(1400);
    stopMotors();
    pos = 0;
```

```
    goto loop10;
  }
  goto loop9;
loop10:
  pos = 0;
}

void exy1() {
loop7:
  if (y < pos) {
    getValues();
    compare();
  } else if (y == pos) {
    delay(1400);
    stopMotors();
    pos = 0;
    goto loop8;
  }
  goto loop7;
loop8:
  pos = 0;
}

void exx1() {
loop9:
  pxy[0] = pos + ox;
  prev[0] = pxy[0];
  if (x < pos) {
    getValues();
```

- **Robot Mapping Code Cont.**

```
loop7:
  pxy[1] = oy + pos;

  prev[1] = pxy[1];
  if (y > pos) {
    getValues();
    compare();
  } else if (y == pos) {
    delay(1400);
    stopMotors();
    pos = 0;
    goto loop8;
  }
  goto loop7;
loop8:
  pos = 0;
}

void exx() {
loop9:
  pxy[0] = ox + pos;
  prev[0] = pxy[0];
  if (x > pos) {
    getValues();
    compare();
  } else if (x == pos) {
    delay(1400);
    stopMotors();
    pos = 0;
```

```
  prev[0] = pxy[0];
  if (x < pos) {
    getValues();
    compare();
  } else if (x == pos) {
    delay(1400);
    stopMotors();
    pos = 0;
    goto loop10;
  }
  goto loop9;
loop10:
  pos = 0;
}


/*
******* Motor Guide **********
*                            *
*    High: Right Motor       *
*       pin1 -> Reverse      *
*       pin2 -> Forward      *
*                            *
*    High: Left Motor        *
*       pin1 -> Forward      *
*       pin2 -> Reverse      *
*                            *
****************************
*/
```

- **Robot Mapping Code Cont.**

```
void moveForward() {
  digitalWrite(leftMotorPin1, HIGH);
  digitalWrite(leftMotorPin2, LOW);

  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);

  analogWrite(enableRightMotor, abs(220));
  analogWrite(enableLeftMotor, abs(220));
}

void moveRight() {
  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);

  digitalWrite(leftMotorPin1, LOW);
  digitalWrite(leftMotorPin2, HIGH);

  analogWrite(enableRightMotor, abs(200));
  analogWrite(enableLeftMotor, abs(200));
}

void moveLeft() {
  digitalWrite(rightMotorPin1, HIGH);
  digitalWrite(rightMotorPin2, LOW);

  digitalWrite(leftMotorPin1, HIGH);
```

- **Robot Mapping Code Cont.**

```
  digitalWrite(rightMotorPin2, LOW);

  digitalWrite(leftMotorPin1, HIGH);
  digitalWrite(leftMotorPin2, LOW);

  analogWrite(enableRightMotor, abs(200));
  analogWrite(enableLeftMotor, abs(200));
}

void stopMotors() {
  digitalWrite(leftMotorPin1, LOW);
  digitalWrite(leftMotorPin2, LOW);

  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, LOW);

  analogWrite(enableRightMotor, 0);
  analogWrite(enableLeftMotor, 0);
}

void moveLeft180() {
  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);

  digitalWrite(leftMotorPin1, LOW);
  digitalWrite(leftMotorPin2, HIGH);

  analogWrite(enableRightMotor, abs(250));
  analogWrite(enableLeftMotor, abs(250));
```

- **Robot Mapping Code Cont.**

```
  analogWrite(enableRightMotor, abs(250));
  analogWrite(enableLeftMotor, abs(250));
  delay(5500);
}

void moveRight180() {
  digitalWrite(rightMotorPin1, HIGH); //Right
  digitalWrite(rightMotorPin2, LOW);

  digitalWrite(leftMotorPin1, HIGH);  //Left
  digitalWrite(leftMotorPin2, LOW);

  analogWrite(enableRightMotor, abs(230));
  analogWrite(enableLeftMotor, abs(230));
  delay(9000);
}

void moveLeft90() {
  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);

  digitalWrite(leftMotorPin1, LOW);
  digitalWrite(leftMotorPin2, HIGH);

  analogWrite(enableRightMotor, abs(220));
  analogWrite(enableLeftMotor, abs(220));
  delay(5650);    //4600
}
```

- **Robot Mapping Code Cont.**

```
void moveLeft90() {
  digitalWrite(rightMotorPin1, LOW);
  digitalWrite(rightMotorPin2, HIGH);

  digitalWrite(leftMotorPin1, LOW);
  digitalWrite(leftMotorPin2, HIGH);

  analogWrite(enableRightMotor, abs(220));
  analogWrite(enableLeftMotor, abs(220));
  delay(5650);     //4600
}

void moveRight90() {

  digitalWrite(rightMotorPin1, HIGH); //Right
  digitalWrite(rightMotorPin2, LOW);

  digitalWrite(leftMotorPin1, HIGH);  //Left m
  digitalWrite(leftMotorPin2, LOW);

  analogWrite(enableRightMotor, abs(220));
  analogWrite(enableLeftMotor, abs(220));
  delay(5650);  //4650
}
```

5.2 Testing

5.2.1 Testing Implementation

This test report documents the results of testing the Museum Tour Guide Robot and its associated mobile application. The testing covered various functional and non-functional requirements to ensure the system's reliability, performance, and usability. All test cases were executed, and the outcomes were documented to validate that the robot meets the project requirements.

Test Case	Input	Expected Outcome	Obtained Outcome	Pass/Fail	Test On
Robot Navigation	Start command, predefined route	Robot Navigates the Routs Successfully	Robot Navigates the Routs Successfully	Pass	15/1/2024
Stop Command	Stop command from mobile app	Robot Stops Moving	Robot Stops Moving	Pass	20/2/2024
Exhibit Query	"Tell me about Exhibit X"	Robot provides detailed information	Robot provides detailed information	Pass	10/3/2024
Content Update	Update exhibit information in backend	Robot provides updated information	Robot provides updated information	Pass	25/3/2024
Control Authority Test	Attempt unauthorized QR Code access	Access is denied	Access is denied	Pass	6/4/2024
Battery Life	Continuous operation without recharging	Robot operates for specified duration	Robot operates for specified duration	Pass	26/5/2024

Table 3 Testing Duratio

5.2.2 Testing Report

This test report is required in order to acquire formal test results, allowing for a quick and detailed review of the results. It's a report that uses data from the patch test office to explain the environmental and operating circumstances, as well as how the test results compare to the test objectives. The test report is quite important, and you must comprehend it. The test report is vital because you need to know whether or not the equipment is ready. It's a text that documents the results of a research study.

- Here are some benefits of testing.

  - Make the UI better and simpler for interaction.
  - Improve The functionality of the application.
  - Enhance the system's performance
  - Make the system very easy to use.

6. Chapter Six

6.1 Conclusion

The museum tour guide robot project represents a significant advancement in the use of robotics and artificial intelligence within cultural institutions. By integrating autonomous navigation, natural language processing, and mobile app connectivity, this project aims to revolutionize the way visitors engage with museum exhibits. The development and deployment of the robot promise to enhance visitor experience by providing interactive, personalized, and accessible tours, ensuring that information is delivered in an engaging and user-friendly manner.

Through careful consideration of both functional and non-functional requirements, the project has been designed to meet the diverse needs of museum visitors, from detailed and multilingual exhibit information to accessibility features that cater to visitors with disabilities. Additionally, the integration with a mobile app offers a flexible and modern approach to interacting with the robot, allowing for remote control and customization of the tour experience.

Overall, the project not only aims to improve the educational and entertainment value of museum visits but also seeks to support museum staff by providing consistent and high-quality tours. The use of data collection and analytics will further enable museums to gain insights into visitor behavior and preferences, fostering continuous improvement in museum services.

6.2 Future Work

As the project progresses, several areas for future work and enhancement have been identified:

1. **Enhanced AI Capabilities:**
   - Further development of the robot's AI to handle more complex and nuanced conversations, improving its ability to understand and respond to a wider range of visitor queries.
   - Incorporation of machine learning algorithms to enable the robot to learn from interactions and improve its performance over time.

2. **Expanded Language Support:**
   - Increasing the number of languages supported by the robot to cater to a more diverse international audience.
   - Implementing real-time translation features to facilitate multilingual group tours.
3. **Advanced Interaction Methods:**
   - Exploring additional modes of interaction such as gesture recognition, touch screens, and augmented reality (AR) to provide more immersive and intuitive user experiences.
   - Integrating virtual reality (VR) elements to offer virtual tours for remote visitors or to enhance physical tours with immersive experiences.
4. **Integration with IoT and Smart Museum Technologies:**
   - Connecting the robot to Internet of Things (IoT) devices within the museum, such as smart exhibits, lighting, and climate control systems, to create a more interactive and responsive environment.
   - Developing features that allow the robot to interact with other smart devices, providing a cohesive and integrated visitor experience.
5. **Enhanced Data Analytics:**
   - Expanding the data analytics capabilities to include predictive analytics, helping museums anticipate visitor needs and optimize exhibit layouts and tour paths.
   - Implementing advanced data visualization tools to present insights in a more accessible and actionable format for museum staff.
6. **Pilot Programs and User Feedback:**
   - Conducting pilot programs in various types and sizes of museums to gather extensive user feedback and refine the robot's functionality and performance.
   - Engaging with diverse visitor groups, including children, seniors, and people with disabilities, to ensure the robot meets a wide range of needs and preferences.
7. **Long-term Maintenance and Support:**
   - Establishing a robust maintenance and support framework to ensure the long-term reliability and performance of the robots.
   - Providing training programs for museum staff to effectively manage and troubleshoot the robot.
8. **Scalability and Customization:**
   - Developing modular components and customizable features to allow museums of different sizes and themes to tailor the robot's capabilities to their specific requirements.
   - Creating a scalable deployment model to facilitate the adoption of the technology across multiple museums globally.

6.3 References

[1] Google Developers. (n.d.). "Guide to app architecture". Retrieved from https://developer.android.com/jetpack/guide

[2] Brown, R. (2018). "Mastering Android Architecture: Improve your application's design and scalability with Android architecture patterns". Packt Publishing. Retrieved from https://www.packtpub.com/product/mastering-android-architecture/9781786467218

[3] FastAPI. (n.d.). "FastAPI: The modern, fast (high-performance) web framework for building APIs with Python 3.7+ based on standard Python type hints". Retrieved from https://www.fastapi.tiangolo.com/

[4] A. Tyagi, "Better Android Apps Using MVVM with Clean Architecture," [Online]. Available: https://www.toptal.com/android/android-apps-mvvm-with-clean-architecture.

[5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). "Deep Learning". MIT Press. Retrieved from https://www.deeplearningbook.org/

[6] Hilton Hotels. (2016). "Meet Connie, Hilton's Robot Concierge".

[7] Hochreiter, S., & Schmidhuber, J. (1997). "Long short-term memory". Neural Computation, 9(8), 1735-1780. Retrieved from https://www.bioinf.jku.at/publications/older/2604.pdf

[8] JetBrains. (n.d.). "Kotlin Programming Language". Retrieved from https://kotlinlang.org/docs/home.html

[9] Dautenhahn, K., Walters, M. L., Koay, K. L., & Boekhorst, R. (2006). "RoboX: A museum tour guide robot". Proceedings of the 11th International Conference on Human-Computer Interaction (HCI 2006).